COS214 Final Project

TIVIDY WORKFLOW MANAGEMENT SYSTEM

SHELBY BODENSTEIN, HANNAH DIEDRICK, KELLY RADEMEYER, JANKE RALL, LIVIA WEBBER, MICHAEL HARDING, TAUHIR SEPTEMBER

Contents

Task 1: Research 3
Task 2: Scenario and System Definition 4
Part A: Initial Scope of TIVIDY 4
A.1 What is TIVIDY 4
A.2 Main Responsibilities of the workflow manager 4
A.3 System boundaries 5
A.4 Definition vs. Running instance 5
A.5 Parts of the system expected to vary 5
Part B: Selected Scenario 6
B.1 The scenario 6
B.2 Organisation or environment 6
B.3 The process being managed 6
B.4 Participants and roles 6
B.5 Major pieces of work 6
B.6 Important decisions and routing points 7
B.7 Approvals and dependencies 7
B.8 Possible failures and exceptions 8
B.9 External systems 8
B.10 Expected outcome 9
Task 3: Activity & UML Diagrams 10
Activity Diagram 1 – Generic TIVIDY Workflow Execution 10
Activity Diagram 2 – Work Assignment, Completion and Escalation 11
Activity Diagram 3 – Jacaranda Academic Hospital Laboratory Workflow 12
Task 4: Design Patterns 13
4.1 Overview 13
Summaries of each pattern: 13
4.2 Pattern Documentation 14
4.3 How the Patterns Work Together 22
4.4 Coverage of the Expected Variation Points 22
4.5 Trade-offs and Patterns Not Used 23
4.6 Supporting Classes 23
UML Diagram 24
Task 6: Sequence & State diagrams 0
Create a start request 0
Work item lifecycle 1
Determine what happens after a work item completes 3
Build Errors, Causes and Fixes 0
Spelling error in WorkflowCommand.h 0

Include path, filename case and missing header 0
Class defined twice (WorkflowData and Participant) 1
Wrong arguments in BackwardRecovery.cpp 1
Wrong argument type in CancelRequestCommand.cpp 2
Declaration and definition do not match in CommandProcessor.cpp 2
Incomplete type and wrong argument in CompleteThenUpdate.cpp 2
Incomplete type inside a template (ConditionalRouting) 3
Function defined in the header without a class (CompleteThenUpdate.h) 3
Linker error: undefined WorkflowInstance functions 4
Incomplete type WorkItemState in Task.h 4
Incomplete type WorkItemState in AutomatedTask and IntegrationTask 5
What this taught us: 5
Assumptions Made For TIVIDY 6
Testing: DocTest and Integration Testing: 7
Repository Link: 9
Bibliography 10

Task 1: Research

A workflow is the set of tasks that are required to be completed in a specific process. (IBM, 2025) In contrast, a workflow management system defines, manages, and executes these processes in a predetermined order using the computer's representation of the workflow logic. Some examples of well-known software applications are Pegasus, Kepler, and Taverna. (SAP, 2026)

A workflow definition is static, containing the rules for execution created by the designers or administrators of a system, whereas the actual execution, containing data and unique inputs, is described by the workflow instance. (Tmssoftware.com, 2025)

Different aspects in a workflow component are described by different types, such as nodes for activities, directed arrows/lines for transitions, and data records for work items. Work items are routed and assigned based on a set of rules, conditional logic, definitions, or queues. Routing mainly uses sequential, conditional, or parallel routing, or loops and iterators. Assignment can be done by direct user assignment, role/group-based assignment, capacity- or skill-based assignment, or round robin or priority balancing. The primary control gates are approval, rejection, and escalation, which determine the progress a work item will make within the workflow. (Sedha, 2025) Approval will be logged and trigger the next sequential step; rejections will redirect or divert the work item on a remedial path, and escalation manages any delays after being triggered by Time-to-Live (TTL) or Service Level Agreement (SLA) timers. (GeeksforGeeks, 2024)

Managing dependencies is done by using explicit mapping, relationship sequencing, and ownership tracking, and it is necessary to know what must be completed and what data is needed for each step before it can successfully run. (Unstructured, 2026) To avoid data loss and ensure a system remains stable, there needs to be structures in place to handle any errors that may occur. (Atlassian, 2024) There needs to be a sense of consistency throughout the system, which is where idempotency is needed, as well as proper order management; however, when errors do inevitably occur, resilience patterns are put in place to minimise and isolate the damage. In essence, a strong architecture uses asynchronous events to break tight dependencies. (GeeksforGeeks, 2024)

Workflows most commonly interact with external systems through data exchange, synchronising processes using a common API to communicate directly and query external software; event-driven triggers can change the processing via webhooks and event listeners; and other middleware connectors. (FlowWright, 2026)

A chronological log captures the events taking place, packages this data along with its metadata into a log or database, kept in a dedicated storage for a set retention period where the information can be queried and retrieved later, and compliance can be verified. (Splashtop, 2024) The workflows can, however, also be monitored in real time on a dashboard with automated alerts and current metrics of each component in the workflow. Reports can then be made from this data by filtering based on status, audit trails based on compliance standards, or intervention controls can be used to change the progress state on a workflow. (Science Direct)

Due to its modular design, the workflow engine can easily be reused using encapsulation, inheritance, and loose coupling. The use of Gang of Four design patterns encourages the modular design of the systems, allowing for nested systems and sub-processes. (Sedha, 2025)

Certain decisions are not necessarily hardcoded in these systems; approval chains, routing and transition rules, and numerical limits may trigger specific paths. (Finio, 2025) If work is already in progress and changes need to be made, there are several strategies used to handle these alterations. First, the currently executing processes are allowed to complete and any new processes will use the updated version. The second option is dynamic migration which only works for smaller changes by swapping the in-progress task to the new version. Thirdly, forward recovery checks the updated rules each time a task completes and automatically reroutes the work item. Lastly, backward recovery sends the work item to re-initiate the workflow from the beginning using the new definition. (team, 2024)

Task 2: Scenario and System Definition

Part A: Initial Scope of TIVIDY

A.1 What is TIVIDY

TIVIDY is a reusable engine that stores workflow definitions, instantiates those definitions as active workflow instances, and manages those instances from inception to outcome. A definition is the static set of rules authored by a designer or administrator, while an instance comprises an execution that holds its data and input. Based on this view, a workflow management system executes a process based on a computer representation of workflow logic.

A.2 Main Responsibilities of the workflow manager

Responsibility	What TIVIDY must be able to do
Definition management	Create, validate, store and version workflow definitions, including their structure, rules and data requirements.
Instance creation and execution	Create independent instances from one definition and move each one forward until it reaches an outcome (completed, rejected, cancelled, failed). A change to one instance must never affect another.
Work structure	Represent work as stages, individual work items and sub-processes. Support sequential and parallel work, and work that only becomes available once its prerequisites are complete. Other parts of the system do not need to know how this structure is stored.
Work-item lifecycle	Track the state of each work item (e.g. created, available, assigned, started, completed, rejected, cancelled, escalated) and only allow operations that are valid in the current state.
Routing and Assignment	Apply routing logic and decide what should happen after work finishes (continue, branch, loop, run in parallel, wait at a join, or end). Choose who or what performs a work item (role-based, skill-based, capacity-based, round-robin).
Approval, rejection, escalation and deadlines	Manage approval, rejection paths, and escalation when a deadline or SLA timer expires without a response.
Dependency and event handling	Track dependencies between work items and respond to internal/external events.
Integration	Communicate with external systems whose interfaces were not designed for TIVIDY, through APIs, webhooks, event listeners, and middleware connectors.
History and recovery	Record actions and maintain a log of events to support undo or rollback recovery when work fails.
Monitoring and reporting	Navigate running structures to produce views (progress, status, delays and bottlenecks), without placing every possible report inside the core model.
Access control and shared resources	Control who may view or perform certain operations and coordinate work that competes for the same limited resource.
Additional work-item behaviour	Allow a work item to gain extra behaviour (validation, priority handling, auditing, notifications, security) without becoming a different kind of task.

A.3 System boundaries

Outside TIVIDY (collaborators and exclusions):

Boundary	How it relates
Workflow designer/administrator	A person who builds definitions and configures rules. Interacts with TIVIDY before and during execution.
Participants	People or automated agents who receive and perform work items. TIVIDY knows their roles and workload but not their personal details.
Managers/monitors	People who view progress, history and reports. They observe and may intervene but do not perform the work.
External systems	Existing business systems (e.g. records, payment, notification services). TIVIDY depends only on the integration boundary, not on their internals.
Out of scope	A user interface, real authentication/identity management, a real database (storage is in memory or simple files), distributed or multi-machine execution, a visual workflow designer, and domain-specific business calculations (these are supplied by the scenario, not built into the engine).

A.4 Definition vs. Running instance

Belongs to the workflow definition	Belongs to a running instance
• Name, description and version • The structure: stages, work items, sub-processes and their nesting • Dependencies and ordering between work (sequential/parallel/conditional) • Routing and transition rules, and loop conditions • Roles required for each piece of work • Which assignment rule applies to each work item • Deadlines and SLA durations • Required data fields and constraints • Which events trigger which work • Which external system each integration step uses • Set of lifecycle states and the transitions allowed between them	• Instance ID and a reference to the definition version it was created from • Workflow data (documents, values) • Current state of every work item • Participants assigned and their current workload • Real start times, due dates and which timers are running • Pending approvals and escalations that have already occurred • History/log of actions and state changes • Results of external calls • The outcome

A.5 Parts of the system expected to vary

Aspects must be designed for independent variation:

1. Routing strategies: Sequential, conditional, parallel, loops
2. Assignment strategies: Direct, role, skill-based, round-robin, priority balancing
3. Approval/escalation rules: Single approver, multi-level chains, TTL/SLA triggers
4. External integrations: Different APIs, webhooks, middleware connectors
5. Notification mechanisms: Email, SMS, dashboard alerts, webhooks
6. Persistence mechanisms: In-memory, relational DB, NoSQL, file-based
7. Monitoring: Different dashboards, metrics, report formats
8. Error/resilience patterns: Retry, circuit breaker, fallback, compensation
9. Work item types: Human tasks, automated tasks, sub-processes, composite tasks
10. Version change strategies: Complete-then-update, dynamic migration, forward/backward recovery

Part B: Selected Scenario

B.1 The scenario

The chosen scenario is a hospital laboratory test processing: a clinician orders blood tests for a patient, a specimen is collected and analysed, the results are validated, and a report is released.

B.2 Organisation or environment

Jacaranda Academic Hospital is a fictional public teaching hospital in Pretoria. Its pathology laboratory processes test requests from wards, theatres and outpatient clinics, covering chemistry, haematology and microbiology. Currently, work is coordinated through paper request forms, phone calls and separate instrument screens. This often leads to lost or mislabelled specimens, unclear priorities, slow communication of critical results and a weak audit trail. TIVIDY will link each test request from ordering to the final released report.

B.3 The process being managed

A clinician orders one or more tests. The patient is identified, the request is registered, and the specimen is collected, labelled and transported to the laboratory, where the specimen is checked for acceptability, and a priority is assigned. All tests in the panel are run in parallel on an available analyser. Results are reviewed by the automated check, a technologist and a pathologist where applicable. Critical values are phoned or messaged to the clinician and must be acknowledged within a specified time. Finally, the authorised report is then released to the hospital record, and the request is closed.

B.4 Participants and roles

Participant/role	Type	Responsibility in the process
Clinician	External person	Orders tests, receives results, and must acknowledge critical results.
Phlebotomist/ward nurse	External role	Verifies patient identity at the bedside, collects and labels the specimen.
Porter/courier	External role	Transports specimens to the laboratory.
Reception clerk	Internal role	Receives specimens, checks labelling and quality, registers them in the lab.
Medical technologist	Internal role	Runs tests, reviews validated results, handles repeats and instrument problems.
Pathologist	Internal role	Reviews flagged or critical results and authorises specialised reports.
Laboratory manager	Internal role	Receives final-level escalations, oversees resources and turnaround performance.
Ward charge nurse/on-call consultant	External role	Next recipients when a critical result is not acknowledged in time.
Automated agents	System participants	Run automatic checks, drive instrument interfaces, print labels and send notifications.

Shared resources that work items compete for: analysers (one or more per discipline), technologist time at each bench, and the limited volume of each specimen.

B.5 Major pieces of work

Work is organised as stages that include work items and sub-processes.

- Stage 1, Order and registration: receive test order, verify patient identity (hospital record), register request, print specimen labels.
- Stage 2, Specimen collection and transport: collect and label specimen, transport to laboratory.
- Stage 3, Laboratory reception: receive specimen, check specimen quality, assign priority (STAT, urgent or routine). A rejected specimen goes to the recollection path.
- Stage 4, Analysis (parallel panels): sub-process per panel required (chemistry, haematology, microbiology): prepare the sample; book the analyser, run the test, and capture the result. Some tests are sent to an external reference laboratory.
- Stage 5, Validation and review: automatic validation by reference ranges, previous-result comparison, and quality control status, then review by technologist, repeat test if required, review by pathologist if flagged.
- Stage 6, Critical result communication: notify clinician, wait for acknowledgement, escalate if no response.
- Stage 7, Authorisation and reporting: authorise results, release report to the hospital record, notify clinician, send billing information, archive.
- Recollection path: notify the ward, request new specimen, wait for it to arrive and return to Stage 3.

B.6 Important decisions and routing points

Decision	Routing result
Does the patient identity on the specimen match the order?	Yes: continue No: reject the specimen and start the recollection path
Is the specimen acceptable (correctly labelled, enough volume, not haemolysed or clotted)?	Yes: continue No: recollection path
What is the request priority?	STAT, urgent or routine selects the queue position and the turnaround deadline.
Which panels does this request need?	Creates one parallel analysis branch per panel; send-out tests go to the external laboratory.
Is a suitable analyser available?	Yes: reserve it No: wait, use another analyser, or run the test manually
Was quality control done for the analyser run to pass?	Yes: results may be validated No: hold results, rerun after corrective action
Do the results pass automatic validation?	Yes and normal: technologist authorises No: technologist review
Is any result a critical value?	Yes: critical result communication No: continue to authorisation
Does an abnormal result need a repeat?	Yes: repeat the test (loop, limited number of times) No: continue
Does this result need pathologist review?	Yes (flagged, critical or specialised test): pathologist review No: technologist authorisation is enough
Was the critical result acknowledged within the time limit?	Yes: record and continue No: escalate to the next person

B.7 Approvals and dependencies

Authorisation levels (rules and limits are configurable data, not hard-coded):

- Normal results that pass automatic validation: technologist authorises.
- Abnormal, flagged or critical results, and specialised tests: pathologist authorises.

- A rejected authorisation sends the result back for repeat or review with a reason.

Escalation of critical results: For example, upon receipt of a critical result, the clinician must acknowledge it within 15 minutes. Otherwise, it will be escalated to the ward charge nurse, on-call consultant, and laboratory manager, with a deadline at each level.

Turnaround targets: For example, turnaround targets are STAT in 1 hour, urgent in 4 hours, and routine in 24 hours. Work nearing its target is escalated in the queue.

Dependencies:

- The specimen must be accepted before starting the analysis.
- The panel analysis proceeds in parallel, and the final report is held until all panels are complete (a preliminary report may be issued for tests with long durations).
- Results validation waits for quality control clearance of the analyser run.
- Repeat tests wait for enough remaining specimen volume.
- Authorisation depends on validation, and report release follows authorisation.
- Critical results must be acknowledged (or fully escalated) before the request can be closed.

B.8 Possible failures and exceptions

Failure/exception	Expected handling
Specimen mislabelled, insufficient or haemolysed	Reject it and start the recollection path, record the reason
Recollection not received by the deadline	Escalate to the ward, close the request as rejected if still not received
Analyser breakdown or no analyser available	Move the work item to another analyser or run it manually, notify the technologist.
Quality control failure	Hold all results from that run, rerun after corrective action, and record the event.
Result fails validation or looks implausible	Send to technologist review, repeat the test if volume allows.
Critical result not acknowledged	Escalate step by step up the chain until someone acknowledges it.
Instrument interface or hospital record unavailable	Retry a limited number of times, then create a manual work item for a person.
Courier delay	Escalate by priority - STAT items are flagged to the lab manager.
Clinician cancels or changes the order	Cancel active work items, release reserved analysers, and keep a record of completed work.
External reference laboratory is late	Send a reminder, escalate to the lab manager, and release other results as a partial report.

B.9 External systems

External system	Used for
Hospital information system	Verifying patient identity and receiving the released report
Analyser/instrument interface	Sending test instructions to analysers and receiving results and quality control status
Label printing service	Printing specimen labels and barcodes
Messgae/email gateway	Notifying clinicians and escalation recipients, especially for critical results
Billing/medical aid system	Receiving billing information once results are reported

External system	Used for
External reference laboratory	Performing tests the hospital laboratory cannot and returning the results

B.10 Expected outcome

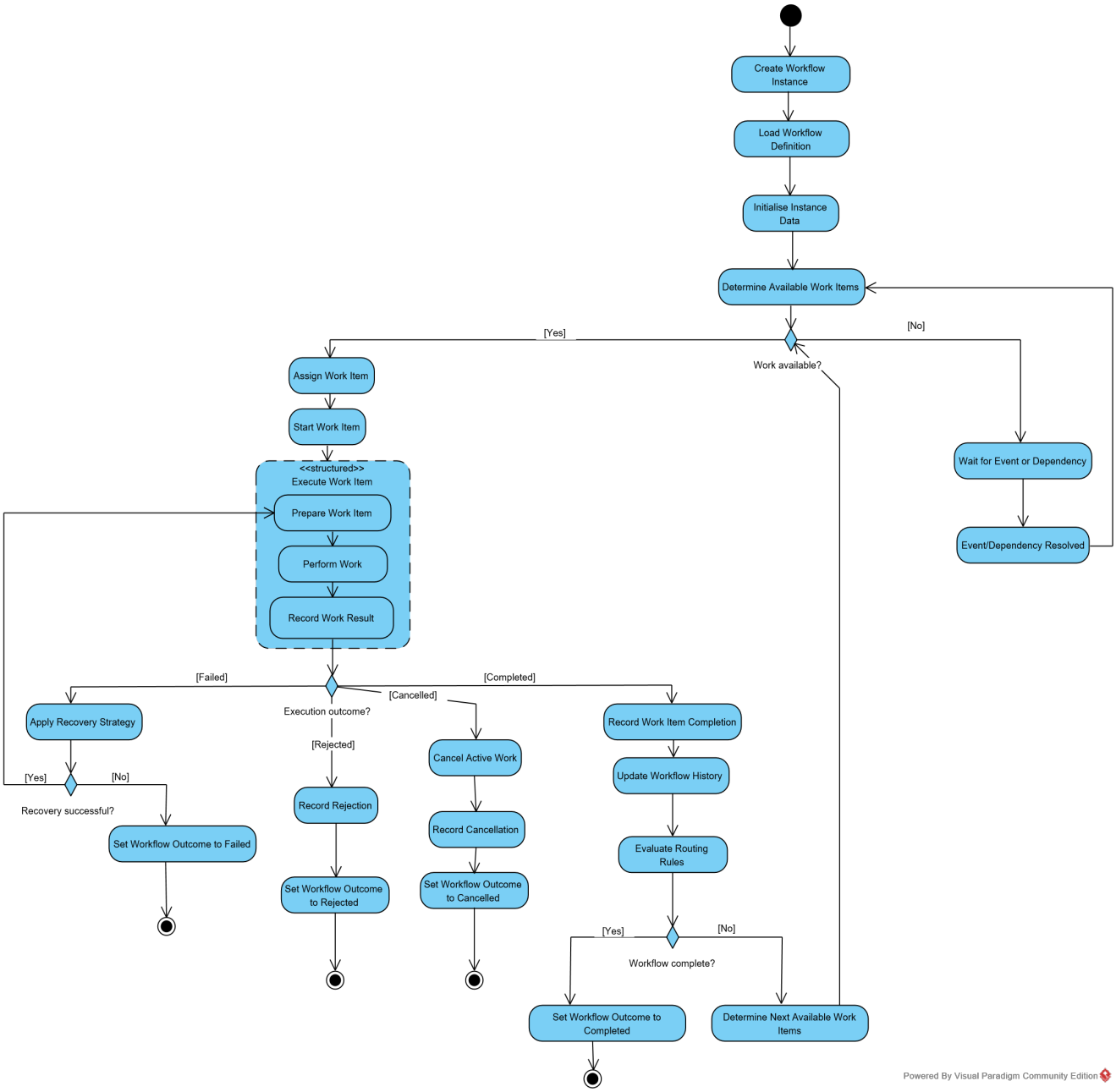
Each test request ends in one outcome:

- Reported (all results authorised and released to the hospital record)
- Rejected (specimen unsuitable or no valid recollection)
- Cancelled (order withdrawn)

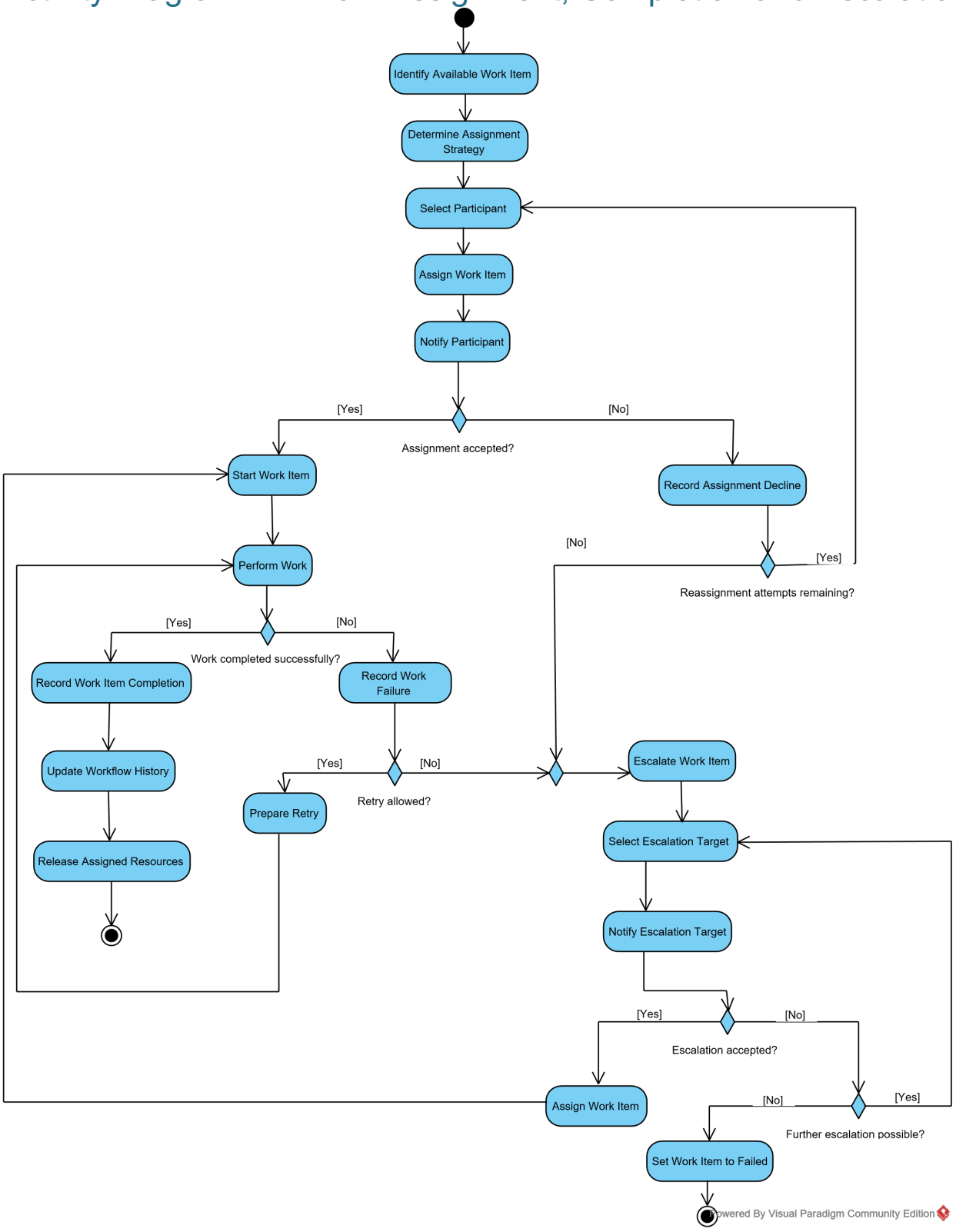
In the end, the clinician will have the report, all critical results will have a recorded acknowledgement, and there will be a complete chain of actions and state changes for each specimen available for audit. In addition, monitoring data can be reported to the laboratory manager, including turnaround time by priority, queue lengths, analyser utilisation, escalations, and rejection rates.

Task 3: Activity & UML Diagrams

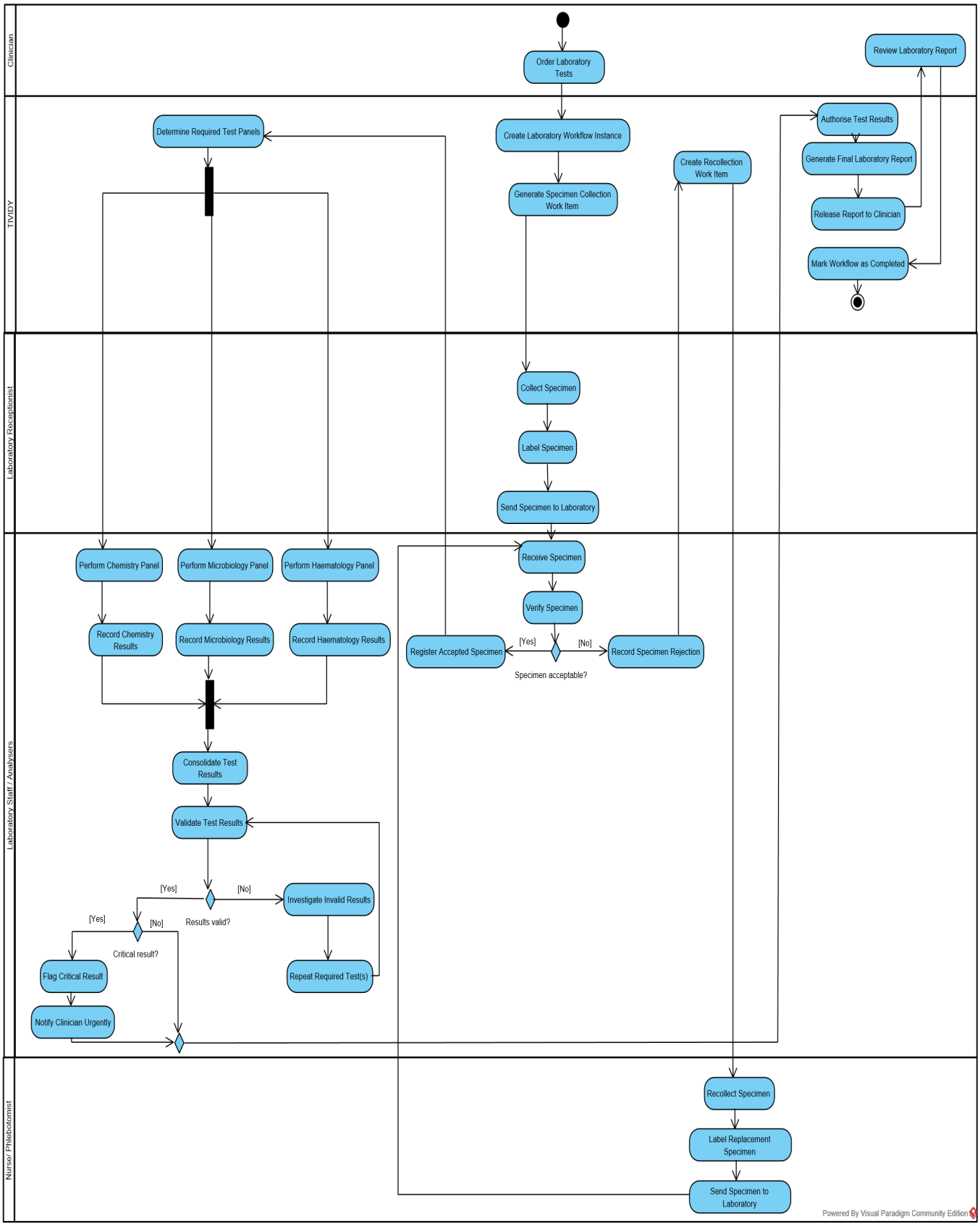
Activity Diagram 1 – Generic TIVIDY Workflow Execution



Activity Diagram 2 – Work Assignment, Completion and Escalation



Activity Diagram 3 – Jacaranda Academic Hospital Laboratory Workflow



Task 4: Design Patterns

4.1 Overview

TIVIDY uses 15 Gang of Four patterns. Each pattern is there because of a force in the workflow problem that was identified in the research and the scope and scenario. Every pattern is documented with the five items the practical asks for: the design problem it solves, the classes that fill the GoF roles, how those classes collaborate, what is expected to vary, and why the pattern is appropriate.

The patterns belong to the generic engine. The Jacaranda laboratory scenario appears only as the data and director code that configure the engine (e.g. LabWorkflowDirector), and as examples inside the tables.

Collaboration group	Patterns	How they work together
Work structure	Composite, Iterator, Visitor, Prototype, Builder	A definition is built (Builder) as a tree of nodes (Composite), copied for each request (Prototype), walked in different ways (Iterator) and reported on (Visitor).
Work behaviour	State, Decorator, Proxy	A single work item has a lifecycle (State), extra behaviour (Decorator) and access control (Proxy).
Actions and recovery	Command, Memento	Every action is an object that can be recorded and undone (Command) and risky steps are checkpointed (Memento).
Rules and decisions	Strategy, Chain of Responsibility	Routing, assignment, version change and resilience rules (Strategy) and escalation and approval levels (Chain of Responsibility) are configurable.
Events and surroundings	Observer, Mediator, Adapter	Parts of the system react to events (Observer), share limited resources (Mediator) and talk to outside systems (Adapter).

Summaries of each pattern:

	Pattern	Where it applies in TIVIDY
1	Composite	Stages, sub-processes and work items as one tree
2	Iterator	Traversing the tree; finding ready items
3	Visitor	Progress, bottleneck and turnaround reports
4	Prototype	Creating independent instances from a definition
5	Builder	Constructing and validating definitions
6	State	Work-item lifecycle
7	Command	Recorded, undoable actions
8	Memento	Checkpoints and rollback of an instance
9	Decorator	Validation, priority, notification, auditing on items
10	Proxy	Role-based access to work items
11	Strategy	Routing, assignment, version change, resilience
12	Chain of Responsibility	Critical-result escalation; authorisation levels
13	Observer	Reacting to workflow and external events
14	Mediator	Allocating analysers, bench time and specimen volume
15	Adapter	Connecting the six external systems

4.2 Pattern Documentation

Composite

Design problem	Work in TIVIDY is nested: a test request has stages, a stage holds work items and panel sub-processes (chemistry, haematology, microbiology), and those hold further items. The engine must start, cancel, query and complete a single item, a stage or a whole subprocess in the same way, without knowing how the structure is stored.
GoF participants (role : TIVIDY class)	• Component: WorkNode (abstract) • Leaf: WorkItem (abstract), with HumanTask and AutomatedTask • Composite: WorkContainer (abstract), with Stage and SubProcess
Collaboration	A caller invokes start(), cancel() or isComplete() on a WorkNode. A WorkContainer forwards the call to its children in the order given by its RoutingStrategy (pattern 11) and is complete when that strategy says so, for example when every panel SubProcess in Stage 4 has finished (the join). Leaves do the actual work. Only WorkContainer has add() and remove(), so a leaf cannot be given children. A container owns its children (composition).
What varies	The shape and depth of the work structure, and the kinds of work item (human, automated, and any future type) without changing containers.
Why appropriate	Without Composite, "clinician cancels the order" would need code that walks stages, sub-processes and items and checks the type of each node, which the non-negotiable rules forbid. With Composite it is one cancel() call on the root. SubProcess differs from Stage because it has its own data scope and its own outcome, so a recollection or a panel can be handled as a unit.

Iterator

Design problem	Different parts of the engine need to walk the same structure in different ways: cancellation visits every active node, the dispatcher needs only items that are ready (all prerequisites completed, "the specimen must be accepted before starting the analysis"), and display needs level order. None of them should know whether children are held in a vector, map or list.
GoF participants (role : TIVIDY class)	• Iterator: WorkNodeIterator (hasNext(), next()) • Concrete iterators: DepthFirstIterator, LevelOrderIterator, ReadyItemIterator • Aggregate: WorkNode (createIterator()) • Concrete aggregate: WorkContainer (builds the iterators; a leaf returns a one-element iterator)
Collaboration	An iterator keeps its own traversal position (a stack or queue of nodes) and reads children only through the container's public interface. ReadyItemIterator skips any WorkItem whose entries in the DependencySet are not yet complete. After each completion the DependencyResolver (pattern 13) loops over a ReadyItemIterator and makes the newly ready items available.
What varies	The traversal order and filter, and how a container stores its children.
Why appropriate	"Available once prerequisites are complete" is defined in exactly one place instead of being repeated in cancellation, dispatch and reporting. A change from a vector to a graph of dependencies changes no caller.

Visitor

Design problem	The laboratory manager needs progress, delays and bottlenecks, turnaround by priority, analyser utilisation, escalations and rejection rates. These are many, change often and only read the structure. Placing each one as a method on every node class would fill the core model with reporting code, which the practical specifically warns against.
GoF participants (role : TIVIDY class)	• Visitor: WorkNodeVisitor (visit(Stage), visit(SubProcess), visit(WorkItem)) • Concrete visitors: ProgressVisitor, BottleneckVisitor, TurnaroundVisitor, DefinitionValidationVisitor

	• Element: WorkNode (accept(visitor)) • Concrete elements: Stage, SubProcess, WorkItem (decorators forward accept to the item they wrap)
Collaboration	ReportService creates a visitor, walks the structure with a DepthFirstIterator and calls accept on each node. The node calls back the matching visit overload (double dispatch) and the visitor accumulates its result, for example time spent waiting per item for BottleneckVisitor. DefinitionValidationVisitor is reused by the Builder (pattern 5) to check a definition before it is published.
What varies	Which reports and checks exist. A new report is a new visitor and no node class changes.
Why appropriate	Reports are the part of TIVIDY most likely to grow. The usual drawback of Visitor, that adding a node type forces every visitor to change, is limited here because the interface has only three overloads: a new kind of task inherits visit(WorkItem).

Prototype

Design problem	One definition creates many independent running instances, and a change to one instance must never affect another. Running instances must also keep the definition version they started with, and a new version is normally a modified copy of the previous one.
GoF participants (role : TIVIDY class)	• Prototype: WorkNode (clone()) • Concrete prototypes: Stage, SubProcess, HumanTask, AutomatedTask, WorkItemDecorator subclasses (each clones the item it wraps) • Client: InstanceManager
Collaboration	InstanceManager takes the template tree held by a published WorkflowDefinition and calls clone() on its root. The deep copy is placed in a new WorkflowInstance with its own identifier, WorkflowData, history and a reference to the definition version. Runtime state (item states, assignees, timers) exists only in the copy. A new definition version is created by cloning the old template and then editing the copy through the Builder.
What varies	The concrete node types being copied; InstanceManager does not know them.
Why appropriate	If instances shared one tree, completing a specimen check for patient A would complete it for patient B. Rebuilding every instance by replaying the Builder would repeat validation and tie instance creation to the scenario code. A virtual clone() gives each request its own copy at low cost and keeps the definition untouched.

Builder

Design problem	A WorkflowDefinition is assembled in many steps: nested stages and items, dependencies, routing and assignment rules, deadlines, data requirements, added behaviours, escalation and approval chains. It must be checked before use (no missing prerequisites, loops have a limit, every required role exists). The Jacaranda laboratory workflow must also stay outside the engine, because the scenario must not define TIVIDY's architecture (non-negotiable rule 3).
GoF participants (role : TIVIDY class)	• Builder: WorkflowDefinitionBuilder (beginStage, addTask, beginSubProcess, endContainer, dependsOn, setRouting, setAssignment, setDeadline, requireData, decorateWith, build) • Concrete builder: DefinitionBuilder • Director: LabWorkflowDirector (scenario package, builds the Jacaranda process) and DefinitionFileReader (builds from a configuration file) • Product: WorkflowDefinition
Collaboration	A director calls the builder steps in order. The builder keeps a stack of open containers so that nesting follows beginStage ... endContainer. build() runs DefinitionValidationVisitor (pattern 3) and returns a versioned definition that is no longer editable. Strategies, decorators and chain handlers are chosen by key and attached during construction.

What varies	The process being described and where it comes from (code or file). A different hospital or industry supplies a different director and the engine is unchanged.
Why appropriate	Authorisation limits, escalation order and deadlines are data, not code. The Builder is the single place where that data becomes objects, and it keeps the laboratory scenario as one client of TIVIDY rather than part of it.

State

Design problem	A work item moves through created, available, assigned, in progress, on hold, escalated, completed, rejected and cancelled, and only some operations are valid in each state: an item cannot be completed before it is started, a cancelled item cannot be reassigned, and results from a run that failed quality control must be held.
GoF participants (role : TIVIDY class)	• Context: WorkItem • State: WorkItemState (makeAvailable, assign, start, complete, reject, cancel, escalate, hold, release) • Concrete states: CreatedState, AvailableState, AssignedState, InProgressState, OnHoldState, EscalatedState, CompletedState, RejectedState, CancelledState
Collaboration	WorkItem forwards every operation to its current state object. The state either performs the change, by calling WorkItem::setState() with the next state, or refuses with an IllegalTransitionException, which is recorded. Entering a state publishes a WorkItemStateChanged event (pattern 13). CompletedState, RejectedState and CancelledState are final.
What varies	Which operations are legal in which state, and the behaviour attached to each transition.
Why appropriate	Every state comes from the scenario: OnHoldState is a quality-control failure, EscalatedState is an expired deadline, RejectedState is a failed authorisation or unacceptable specimen. Without State, each operation would contain a switch on the current state, which the rules prohibit. The same states are used in the Task 6 State Diagram. The overall result of a request (Reported, Rejected, Cancelled) is derived from its root node and kept as a simple value for now.

Command

Design problem	Actions on work (assign, start, complete, approve, reject, escalate, cancel a request) are triggered by different sources: participants, deadline timers, event listeners and integrations. Each action must be recorded for audit, queued for retry when needed, and undone or compensated where sensible.
GoF participants (role : TIVIDY class)	• Command: WorkflowCommand (execute(), undo(), isUndoable(), describe()) • Concrete commands: AssignCommand, StartCommand, CompleteCommand, ApproveCommand, RejectCommand, EscalateCommand, CancelRequestCommand (a macro command that cancels active items and releases analysers) • Invoker: CommandProcessor • Receivers: WorkItem, WorkflowInstance, ResourceCoordinator • Client: the code that creates the command (participants' requests, DeadlineMonitor, EventTriggerListener)
Collaboration	The client builds a command naming the receiver and its parameters and gives it to CommandProcessor. The processor executes it, appends it with who, when and what to the instance's ActionHistory, and publishes the outcome. undo() applies the inverse operation, for example unassigning an item. Commands that cannot be reversed, such as sending a critical-result message, return false from isUndoable(); they are corrected by a compensating command. Undo and compensation are appended to the history and never delete earlier entries.
What varies	The set of actions, whether each can be undone, and who may trigger them.
Why appropriate	The audit trail required by the scenario is a list of command objects. Without Command, the history would be separate logging code copied into every operation.

Memento

Design problem	Some failures invalidate more than one action. When quality control fails, every result from that analyser run is held and the run repeated; moving an instance to a new definition version by backward recovery restarts it. The instance must return to an earlier consistent state without other classes seeing its internals.
GoF participants (role : TIVIDY class)	• Originator: WorkflowInstance (createSnapshot(), restore()) • Memento: InstanceSnapshot (item states, assignments, data values and timers; readable only by the instance) • Caretaker: RecoveryManager
Collaboration	Before a risky step, such as starting an analyser run or migrating to a new version, RecoveryManager asks the instance for a snapshot and stores it under a checkpoint label without reading it. On failure it passes the snapshot back and the instance restores itself. The audit history is not part of the snapshot: it only grows, and the rollback is logged as its own entry (for example "run rolled back after QC failure").
What varies	What state is captured, when checkpoints are taken and how long they are kept.
Why appropriate	Command undoes one step; Memento restores a whole group of changes in one move. Keeping the snapshot opaque means the recovery code does not change when work items gain new data.

Decorator

Design problem	A work item must be able to gain behaviour without becoming a different kind of task. In the laboratory a result-capture item needs reference-range validation, STAT items need a queue position and turnaround deadline, critical values need a notification, and all items need auditing. Which combination applies differs per item, so subclasses for every combination would multiply quickly.
GoF participants (role : TIVIDY class)	• Component: WorkItem • Concrete components: HumanTask, AutomatedTask • Decorator: WorkItemDecorator (is a WorkItem and holds a WorkItem) • Concrete decorators: ValidationDecorator, PriorityDecorator, NotificationDecorator, AuditDecorator
Collaboration	The Builder wraps items as the definition is created. A call such as complete() passes through the wrappers: ValidationDecorator checks the result against reference ranges and quality-control status before passing it on, PriorityDecorator sets the deadline and queue position on assignment, NotificationDecorator publishes a message when a result is critical, and AuditDecorator records the call. clone() copies the whole chain and accept() is forwarded to the innermost item so a visitor sees each item once.
What varies	Which behaviours an item has and in what order, set in the definition rather than in code.
Why appropriate	The behaviours are optional and combinable, and HumanTask should know nothing about reference ranges or STAT rules. State handles the item's own lifecycle, whereas Decorator adds behaviour around the same operations. Access control is handled by Proxy rather than a decorator.

Proxy

Design problem	Not every participant may do everything: the reception clerk registers specimens, only a pathologist authorises flagged or specialised results, and managers view but do not perform work. The work item itself should not contain permission checks. Identity management is out of scope but the engine must still enforce roles.
GoF participants (role : TIVIDY class)	• Subject: WorkItem • Real subject: HumanTask or AutomatedTask, possibly wrapped by decorators • Proxy: AccessControlProxy • Supporting classes: AccessPolicy (role to permitted operations), Participant, Role
Collaboration	Operations on WorkItem carry the acting Participant. The proxy asks AccessPolicy whether that role may perform the operation on that item. If so it delegates to the real item; if not it refuses and publishes an AccessDenied event for the audit log. The proxy is the outermost wrapper (proxy, then decorators, then item), and outside code is only given proxies.
What varies	The permission rules, which are configuration data.
Why appropriate	This is a protection proxy: it controls access to the object and adds no behaviour of its own, which is why it is not a decorator. The same real item can be used with a different AccessPolicy for another organisation.

Strategy

Design problem	Organisations make different decisions about the same kind of work, and those decisions must not force work-item classes to be rewritten. TIVIDY has four of them: how work is routed after something finishes (sequential, conditional, parallel, loop), who is assigned (direct, role, skill, capacity, round robin), what happens to running instances when a definition changes and how a failed external call is handled.
GoF participants (role : TIVIDY class)	• Routing: context WorkContainer; strategy RoutingStrategy; concrete SequentialRouting, ConditionalRouting, ParallelRouting (fork and join), LoopRouting (with a maximum count) • Assignment: context AssignmentService; strategy AssignmentStrategy; concrete DirectAssignment, RoleAssignment, SkillAssignment, CapacityAssignment, RoundRobinAssignment • Version change: context VersionManager; strategy MigrationStrategy; concrete CompleteThenUpdate, DynamicMigration, ForwardRecovery, BackwardRecovery • Resilience: context IntegrationTask; strategy ResiliencePolicy; concrete RetryPolicy (limited attempts), FallbackPolicy (create a manual work item)
Collaboration	The definition names each strategy by key and the Builder installs the matching object. At run time the context calls it: a container asks nextNodes(completedNode, data), AssignmentService asks selectParticipant(item, candidates), VersionManager asks migrate(instance, newVersion), and IntegrationTask asks onFailure(attempt). Conditional and loop strategies evaluate Condition objects over WorkflowData, so the rule is data.
What varies	All four decisions, per definition and per organisation, without changing containers or tasks.
Why appropriate	In the laboratory, a critical value routes to critical-result communication while a normal result goes to authorisation; microbiology items can be assigned by skill while ward tasks use round robin; a hospital that prefers to finish running requests first picks CompleteThenUpdate. Each of these would otherwise be a growing if/else chain inside the engine.

Chain of Responsibility

Design problem	A critical result must be acknowledged by the clinician within 15 minutes; if not it goes to the ward charge nurse, then the on-call consultant, then the laboratory manager, each with a deadline. Authorisation also has levels: the technologist authorises normal validated results and the pathologist handles abnormal, flagged or specialised ones, with limits that are configurable data. The code raising the request must not know who finally handles it.
GoF participants (role : TIVIDY class)	• Handler (escalation): EscalationHandler (setNext(), handle(EscalationRequest)) • Concrete handlers: EscalationLevel (a role and a deadline, configured as data) and TerminalEscalationLevel (laboratory manager; no next level) • Handler (approval): ApprovalHandler • Concrete handlers: TechnologistApproval, PathologistApproval (each with ApprovalLimits) • Clients: DeadlineMonitor, AuthorisationTask
Collaboration	Escalation: when a DeadlineExpired event arrives, the current level checks whether the result was acknowledged. If so it records the acknowledgement and stops; if not it notifies its role through a connector, starts its own deadline and, when that expires, passes the request to the next level. TerminalEscalationLevel records that the chain is exhausted so the request can be closed. Approval: AuthorisationTask submits the result, TechnologistApproval approves it if it is within limits and otherwise forwards it to PathologistApproval; a rejection returns the result for repeat or review with a reason.
What varies	Who is in each chain, in what order, with what deadlines and limits. The Builder assembles each chain from definition data.
Why appropriate	This is the escalation and approval behaviour that the scenario describes. The engine can add a level, such as a second consultant, by adding data, and the sender stays unaware of the length of the chain.

Observer

Design problem	One occurrence interests several unrelated parts. When validation completes, authorisation must become available, the audit log must record it, the metrics must update and the clinician may need a message. An external event such as a cancelled order must also start new work. New rules and integrations must be addable without redesigning the engine.
GoF participants (role : TIVIDY class)	• Subject: WorkflowEventPublisher (subscribe(type, listener), unsubscribe(), publish(event)) • Observer: WorkflowEventListener (onEvent(event)) • Concrete observers: DependencyResolver, AuditLogger, NotificationListener, MetricsCollector, EventTriggerListener • Event sources: WorkItem, DeadlineMonitor, IntegrationTask, external events
Collaboration	Sources publish typed events (WorkItemStateChanged, DeadlineExpired, CriticalResultFlagged, OrderCancelled, IntegrationFailed, AccessDenied). The publisher calls every listener subscribed to that type. DependencyResolver walks a ReadyItemIterator and makes newly ready items available. EventTriggerListener turns an external event into a command, for example OrderCancelled becomes a CancelRequestCommand. Listeners act through commands so their actions are also audited, and none may depend on the order in which listeners are notified.
What varies	Which parties react to which events, and the event types.
Why appropriate	Completing an item must not contain "then log, then notify, then update the dashboard". Adding an SMS notifier or a new dashboard is a new listener that subscribes, with no change to the publisher or to work items.

Mediator

Design problem	Analysers, technologist time and the remaining volume of each specimen are limited, and many work items compete for them. The scenario needs STAT work to go first, an alternative analyser or manual processing when none is free, and release when work is cancelled or an analyser breaks down. If tasks negotiated directly with each resource and with each other, every task would depend on all resources and all other tasks.
GoF participants (role : TIVIDY class)	• Mediator: ResourceMediator (reserve(), release()) • Concrete mediator: ResourceCoordinator • Colleagues: Resource (abstract) with Analyser, TechnologistBench, SpecimenVolume; and the requesting AutomatedTask and HumanTask
Collaboration	A task asks the coordinator to reserve a resource type with its priority. The coordinator chooses a matching resource and grants it, queues the request in priority order, proposes another analyser, or answers that the test must be run manually. When a task finishes, is cancelled or its analyser fails, the coordinator releases the resource and offers it to the next waiting request, informing the requester through a callback. A repeat test first asks SpecimenVolume whether enough specimen remains.
What varies	The allocation policy and the number and kinds of resources (for example one analyser per discipline).
Why appropriate	Decisions such as "is a suitable analyser available" involve many tasks and many resources at once, so they belong in one place. The mediator is limited to resource coordination so it does not grow into a central controller: routing stays with Strategy and the Composite structure.

Adapter

Design problem	TIVIDY must talk to six outside systems whose interfaces were not designed for it: the hospital information system, analyser interface, label printing service, message gateway, billing system and external reference laboratory. Each uses its own protocol and message format, and a hospital may replace one (for example email with SMS). The engine must depend only on the integration boundary.
GoF participants (role : TIVIDY class)	• Target: ExternalConnector (send(request), isAvailable()) • Adapters: HospitalInfoSystemAdapter, InstrumentInterfaceAdapter, LabelPrinterAdapter, MessageGatewayAdapter, BillingAdapter, ReferenceLabAdapter • Adaptees: the external systems' own interfaces (simulated in the implementation) • Client: IntegrationTask (an AutomatedTask), which finds its connector in ConnectorRegistry
Collaboration	IntegrationTask builds a neutral request from WorkflowData and fetches the connector named in the definition from ConnectorRegistry. The adapter translates the request into the external format, calls the adaptee, and translates the reply back into neutral data. If the call fails, the ResiliencePolicy retries a limited number of times and then publishes IntegrationFailed, which creates a manual work item for a person.
What varies	Which external system is used for a step, and its protocol. Adapters use composition, so each wraps its adaptee.
Why appropriate	The external systems cannot be changed, and TIVIDY's classes must not contain their formats. With adapters, replacing the message gateway means writing one new adapter and changing the connector name in the definition.

4.3 How the Patterns Work Together

The patterns form one design rather than isolated examples. The following sequence follows a single work item, "capture chemistry result", through the engine and shows where each pattern takes part. It can be used as the basis for the Task 6 Sequence Diagrams.

1. Definition. LabWorkflowDirector drives DefinitionBuilder (Builder) to create nested Stage and SubProcess nodes (Composite), choosing the routing and assignment strategies (Strategy), wrapping the item with ValidationDecorator and NotificationDecorator (Decorator) and attaching the escalation and approval chains (Chain of Responsibility). DefinitionValidationVisitor (Visitor) checks the result.
2. Instance. When a test request is ordered, InstanceManager calls clone() on the template (Prototype), producing an independent WorkflowInstance.
3. Becoming available. When the specimen check finishes, WorkItem publishes WorkItemStateChanged (Observer). DependencyResolver walks a ReadyItemIterator (Iterator) and the chemistry item moves to AvailableState (State).
4. Assignment and resources. AssignmentService uses the item's AssignmentStrategy to pick a technologist. The item asks ResourceCoordinator (Mediator) for an analyser.
5. Doing the work. A participant's request reaches the item through AccessControlProxy (Proxy) as a CompleteCommand run by CommandProcessor (Command), after RecoveryManager has taken a snapshot (Memento). The decorators validate the result, and an InstrumentInterfaceAdapter (Adapter) delivers the analyser data.
6. Aftermath. The item enters CompletedState. If a value is critical, CriticalResultFlagged triggers the escalation chain; AuditLogger and MetricsCollector (Observer) record the action, and ProgressVisitor can report on it.

4.4 Coverage of the Expected Variation Points

	Variation point (Task 2, A.5)	Pattern(s)	Remarks
1	Routing strategies	Strategy (RoutingStrategy)	
2	Assignment strategies	Strategy (AssignmentStrategy)	
3	Approval and escalation rules	Chain of Responsibility	
4	External integrations	Adapter	
5	Notification mechanisms	Observer + Adapter	Listener decides when; MessageGatewayAdapter decides how
6	Persistence mechanisms	None of the 15	Handled through plain repository interfaces
7	Monitoring	Visitor	
8	Error and resilience patterns	Strategy (ResiliencePolicy), Command, Memento	Retry and fallback in Strategy; compensation and rollback in Command and Memento
9	Work-item types	Composite, Decorator	Leaf and composite types; added behaviour
10	Version change strategies	Strategy (MigrationStrategy), Memento, Prototype	Backward recovery uses a snapshot; new versions are cloned

4.5 Trade-offs and Patterns Not Used

- Pattern count. The design has 15 patterns against a minimum of 10. If the Task 5 class diagram becomes too large, the first candidates to remove are Proxy (permission checks could be placed in CommandProcessor) and Memento (rollback could rely on Command compensation alone). The remaining 13 would still satisfy the requirement. This should be decided once the class diagram is drawn.
- Persistence (variation point 6). Task 2 places a real database out of scope and uses memory or simple files. DefinitionRepository, InstanceRepository and AuditStore are therefore plain interfaces with in-memory and file implementations. If the team wants a sixteenth pattern here, Abstract Factory (a PersistenceFactory creating a matching family of the three repositories) would fit, but it is not needed for the current scope.
- Template Method. A fixed skeleton for IntegrationTask (check input, send, parse reply, handle failure) is a reasonable candidate, but retry behaviour is already varied through Strategy, so it was not added to avoid a pattern without its own force.
- Facade. A TividyEngine class will exist as the entry point for the designer and monitors. It is not documented as a pattern because it would only forward calls and does not solve a separate design problem here.
- Singleton. Deliberately avoided. Global shared objects such as the event publisher or resource coordinator would make instances depend on hidden shared state, and they would be hard to test. Each is created by the engine and passed to the objects that need it.

Interaction between patterns. Decorator and Proxy both wrap WorkItem, so their order is fixed by the Builder (proxy outermost). Visitor, Prototype and Iterator must all handle decorated items, which is why decorators forward accept() and clone(). These points must be reflected in the Task 5 class diagram.

4.6 Supporting Classes

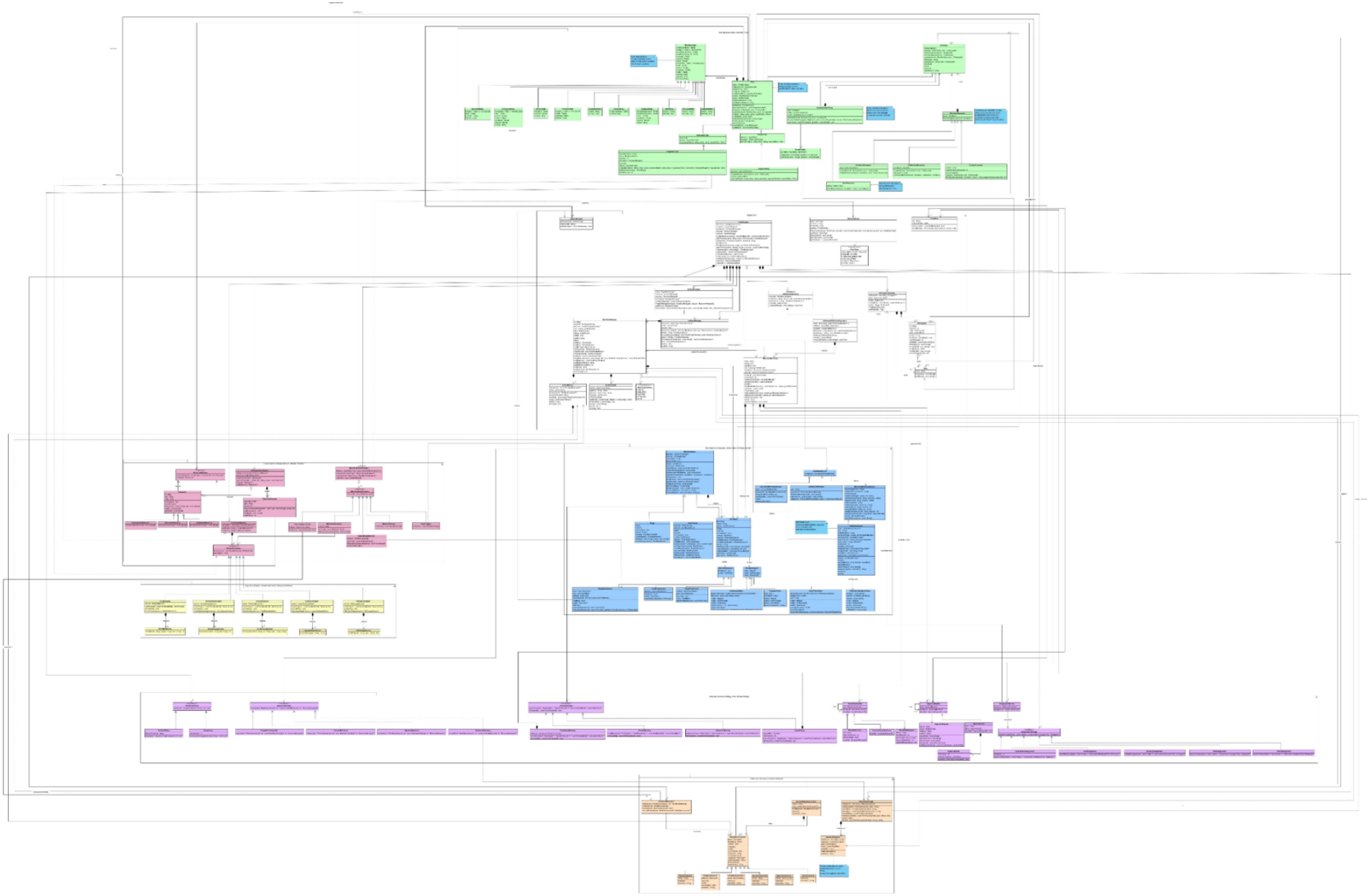
These classes are not GoF participants but appear in the participant tables above, so they must be present in the Task 5 class diagram.

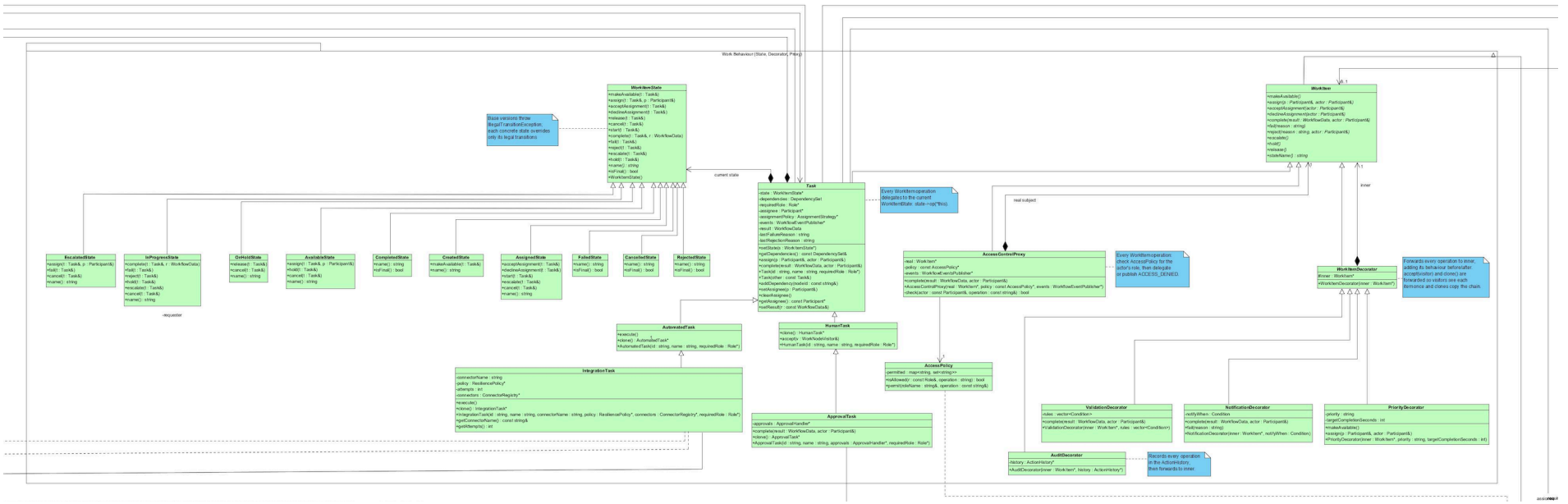
Class	Responsibility	Used by
WorkflowDefinition	Immutable, versioned template: name, version, root WorkNode, rules, data requirements	Builder, Prototype
WorkflowInstance	One running request: id, definition version, cloned tree, WorkflowData, history	Prototype, Memento, Command
InstanceManager	Creates, stores and finds instances	Prototype
WorkflowData	Values and documents carried by an instance (results, priority flags)	Strategy, Decorator
Condition	Evaluates a rule over WorkflowData (e.g. "result is critical")	Strategy
DependencySet	Lists the nodes that must be complete before an item can start	Iterator
Participant, Role	Person or automated agent and the role they hold	Strategy, Proxy
ActionHistory	Append-only record of commands and state changes per instance	Command, Observer
DeadlineMonitor	Timers for turnaround and acknowledgement deadlines	Chain of Responsibility, Observer
ReportService	Runs visitors over an instance and returns report data	Visitor
VersionManager	Applies a MigrationStrategy when a new definition version is published	Strategy, Memento
TividyEngine	Entry point for the designer, participants and monitors	All

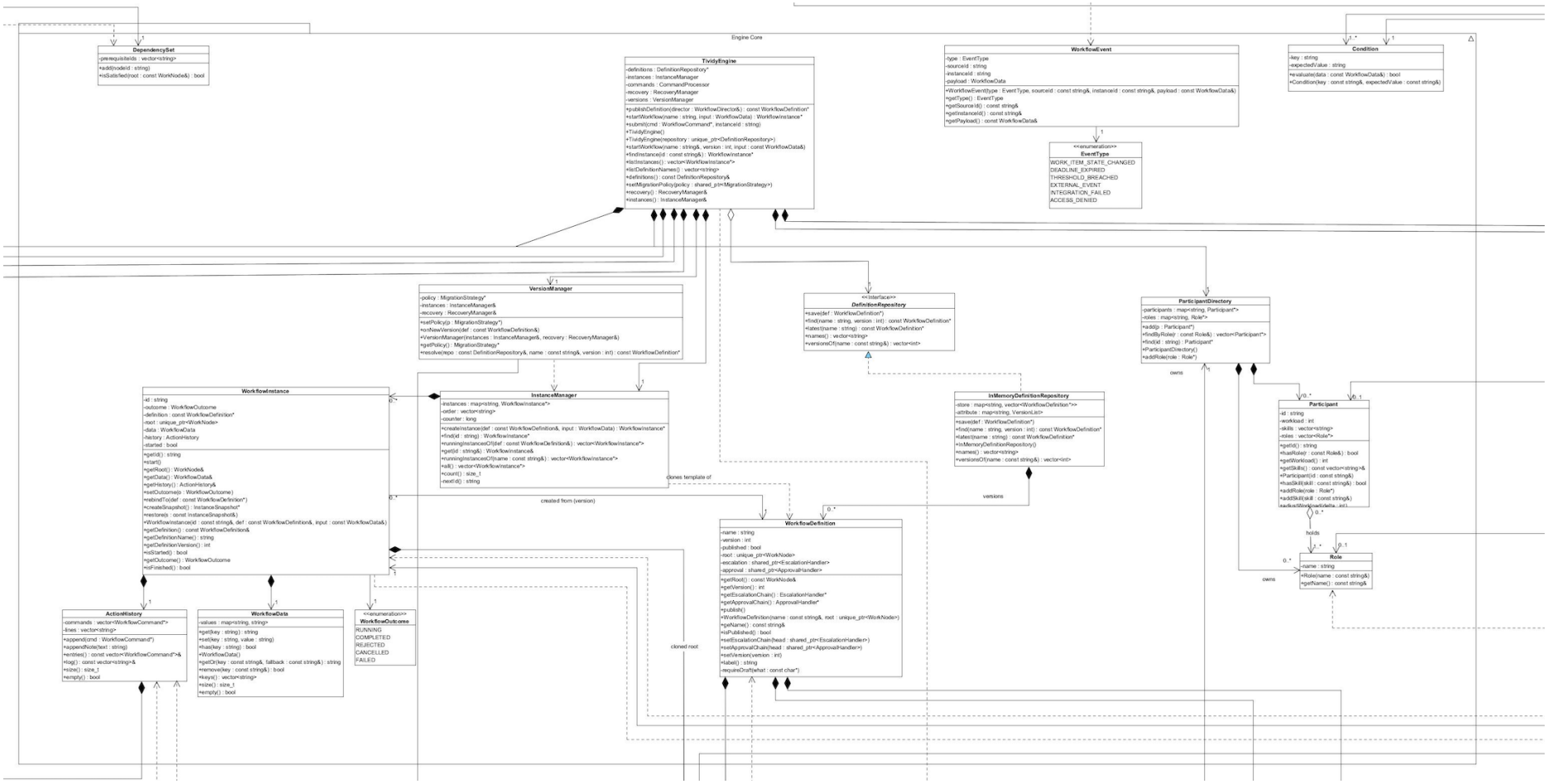
UML Diagram

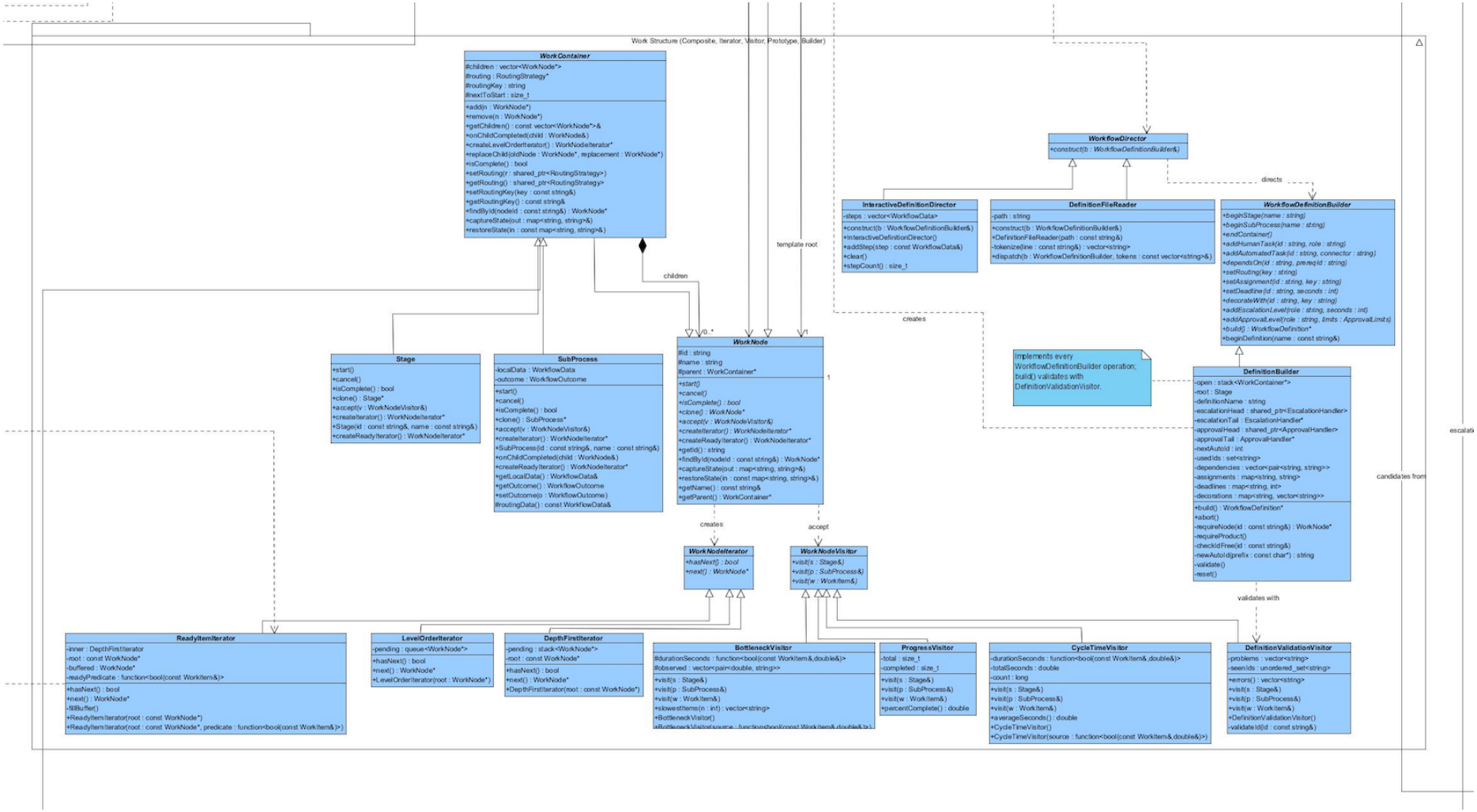
Legend - pattern collaborations (repeated on every figure)

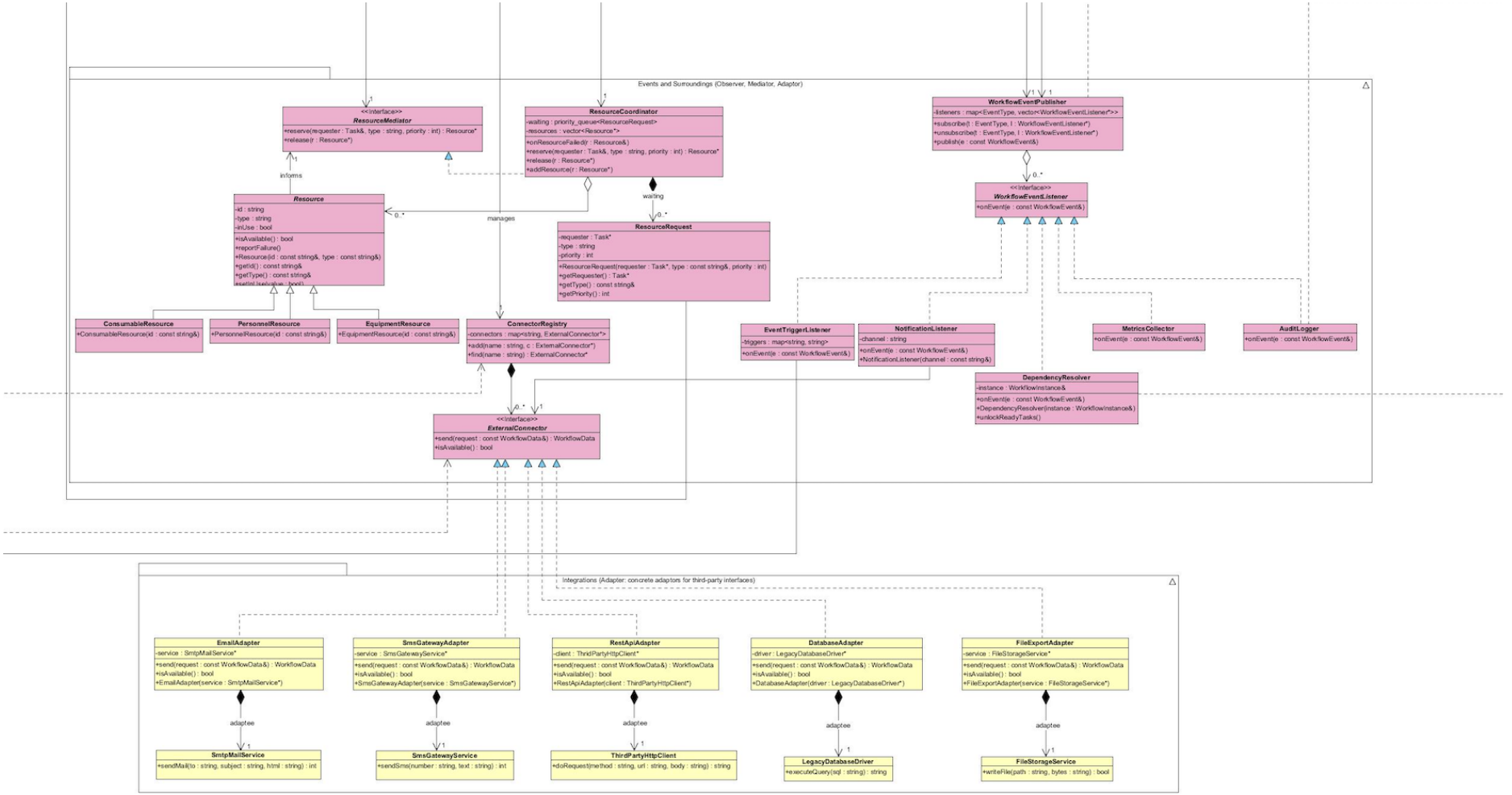
Engine core / supporting classes	no GoF role, or Facade-like entry point
Work structure	Composite, Iterator, Visitor, Prototype, Builder
Work behaviour	State, Decorator, Proxy
Actions and recovery	Command, Memento
Rules and decisions	Strategy, Chain of Responsibility
Events and surroundings	Observer, Mediator, Adapter
Integrations	Concrete Adapters and third-party Adaptees

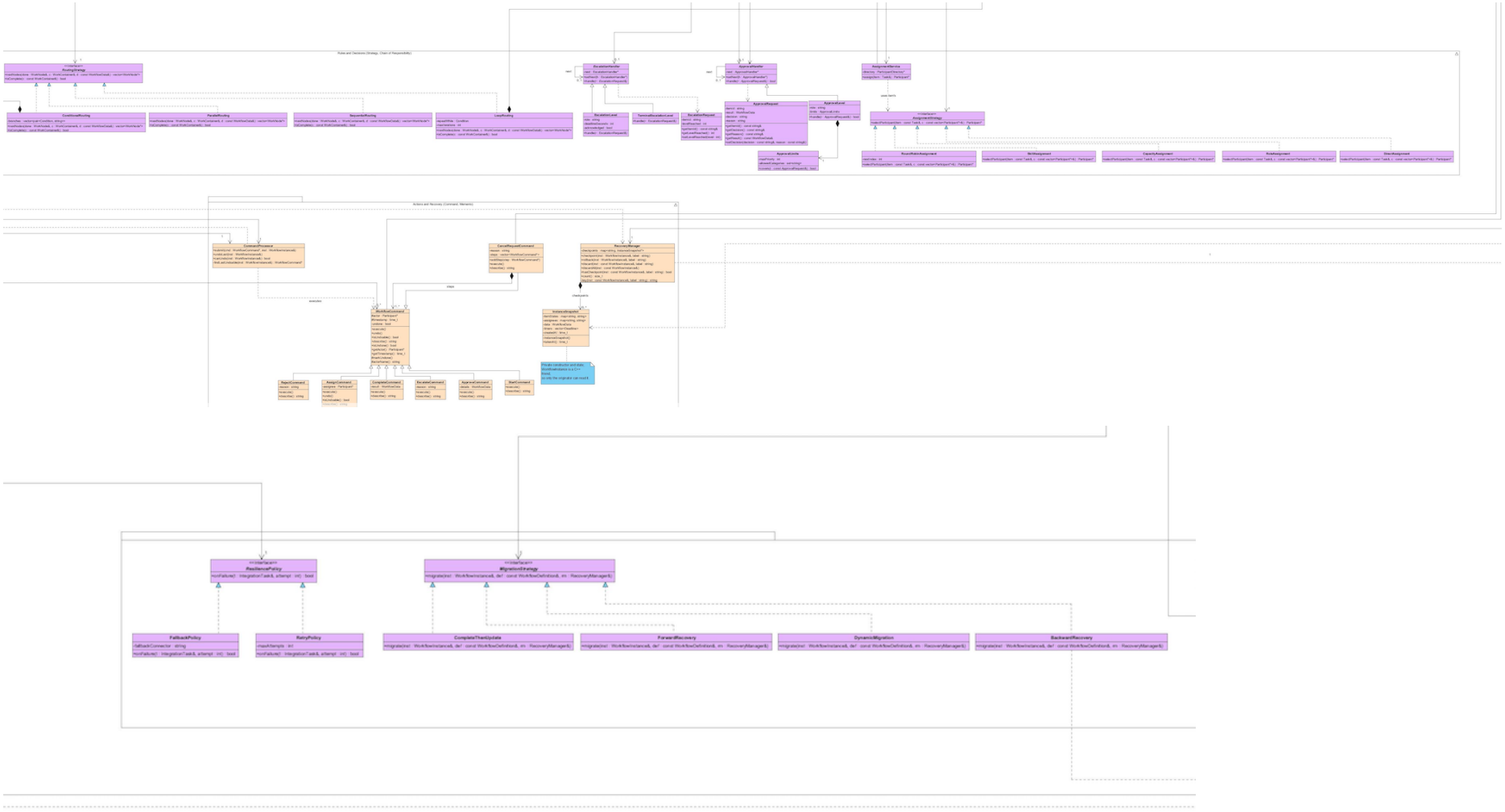



Work Behaviour (State, Decorator, Proxy)
WorkItemState
+makeAvailable() : Task&
+assign(t : Task&, p : Participant&)
+acceptAssignment() : Task&
+declineAssignment() : Task&
+released() : Task&
+cancel() : Task&
+start() : Task&
+complete(r : Task&, r : WorkflowData)
+fail() : Task&
+reject() : Task&
+escalate() : Task&
+hold() : Task&
+name() : string
+isFinal() : bool
#WorkItemState()
Base versions throw IllegalTransitionException; each concrete state overrides only its legal transitions
current state
Task
-state : WorkItemState*
-dependencies : DependencySet
-requiredRole : Role*
-assignee : Participant*
-assignmentPolicy : AssignmentStrategy*
-events : WorkflowEventPublisher*
-result : WorkflowData
-lastFailureReason : string
-lastRejectionReason : string
+setState(s : WorkItemState*)
+getDependencies() : const DependencySet&
+assign(p : Participant&, actor : Participant&)
+complete(result : WorkflowData, actor : Participant&)
+Task(id : string, name : string, requiredRole : Role*)
+Task(other : const Task&)
+addDependency(nodeId : const string&)
+setAssignee(p : Participant&)
+clearAssignee()
+getAssignee() : const Participant*
+setResult(r : const WorkflowData&)
Every WorkItem operation delegates to the current WorkItemState: state->op(*this).
EscalatedState
+assign(t : Task&, p : Participant&)
+fail() : Task&
+cancel() : Task&
+name() : string
InProgressState
+complete(r : Task&, r : WorkflowData)
+fail() : Task&
+reject() : Task&
+hold() : Task&
+escalated() : Task&
+cancel() : Task&
+name() : string
-requester
OnHoldState
+released() : Task&
+cancel() : Task&
+name() : string
AvailableState
+assign(t : Task&, p : Participant&)
+hold() : Task&
+cancel() : Task&
+name() : string
CompletedState
+name() : string
+isFinal() : bool
CreatedState
+makeAvailable() : Task&
+name() : string
AssignedState
+acceptAssignment() : Task&
+declineAssignment() : Task&
+start() : Task&
+escalated() : Task&
+cancel() : Task&
+name() : string
FailedState
+name() : string
+isFinal() : bool
CancelledState
+name() : string
+isFinal() : bool
RejectedState
+name() : string
+isFinal() : bool
AutomatedTask
+execute()
+clone() : AutomatedTask*
+AutomatedTask(id : string, name : string, requiredRole : Role*)
HumanTask
+clone() : HumanTask*
+accept(v : WorkNodeVisitor&)
+HumanTask(id : string, name : string, requiredRole : Role*)
IntegrationTask
-connectorName : string
-policy : ResiliencePolicy*
-attempts : int
-connectors : ConnectorRegistry*
+execute()
+clone() : IntegrationTask*
+IntegrationTask(id : string, name : string, connectorName : string, policy : ResiliencePolicy*, connectors : ConnectorRegistry*, requiredRole : Role*)
+getConnectorName() : const string&
+getAttempts() : int
ApprovalTask
-approvals : ApprovalHandler*
+complete(result : WorkflowData, actor : Participant&)
+clone() : ApprovalTask*
+ApprovalTask(id : string, name : string, approvals : ApprovalHandler*, requiredRole : Role*)
AccessControlProxy
-real : WorkItem*
-policy : const AccessPolicy*
-events : WorkflowEventPublisher*
+complete(result : WorkflowData, actor : Participant&)
+AccessControlProxy(real : WorkItem*, policy : const AccessPolicy*, events : WorkflowEventPublisher*)
-check(actor : const Participant&, operation : const string&) : bool
real subject
Every WorkItem operation: check AccessPolicy for the actor's role, then delegate or publish ACCESS_DENIED.
AccessPolicy
-permitted : map<string, set<string>>
+isAllowed(r : const Role&, operation : string) : bool
+permit(roleName : string&, operation : const string&)
WorkItem
+makeAvailable()
+assign(p : Participant&, actor : Participant&)
+acceptAssignment(actor : Participant&)
+declineAssignment(actor : Participant&)
+complete(result : WorkflowData, actor : Participant&)
+fail(reason : string)
+reject(reason : string, actor : Participant&)
+escalate()
+hold()
+release()
+state(name) : string
0..1
inner
WorkItemDecorator
#inner : WorkItem*
#WorkItemDecorator(inner : WorkItem*)
Forwards every operation to inner, adding its behaviour before/after. accept(visitor) and clone() are forwarded so visitors see each item once and clones copy the chain.
ValidationDecorator
-rules : vector<Condition>
+complete(result : WorkflowData, actor : Participant&)
+ValidationDecorator(inner : WorkItem*, rules : vector<Condition>)
NotificationDecorator
-notifyWhen : Condition
+complete(result : WorkflowData, actor : Participant&)
+fail(reason : string)
+NotificationDecorator(inner : WorkItem*, notifyWhen : Condition)
PriorityDecorator
-priority : string
-targetCompletionSeconds : int
+makeAvailable()
+assign(p : Participant&, actor : Participant&)
+PriorityDecorator(inner : WorkItem*, priority : string, targetCompletionSeconds : int)
AuditDecorator
-history : ActionHistory*
+AuditDecorator(inner : WorkItem*, history : ActionHistory*)
Records every operation in the ActionHistory, then forwards to inner.


Engine Core
DependencySet
-prerequisiteIds : vector<string>
+add(nodeId : string)
+isSatisfied(root : const WorkNode&) : bool
TividyEngine
-definitions : DefinitionRepository*
-instances : InstanceManager
-commands : CommandProcessor
-recovery : RecoveryManager
-versions : VersionManager
+publishDefinition(directory : WorkflowDirector&) : const WorkflowDefinition*
+startWorkflow(name : string, input : WorkflowData) : WorkflowInstance*
+submit(cmd : WorkflowCommand*, instanceId : string)
+TividyEngine()
+TividyEngine(repository : unique_ptr<IDefinitionRepository>)
+startWorkflow(name : string&, version : int, input : const WorkflowData&)
+findInstance(id : const string&) : WorkflowInstance*
+listInstances() : vector<WorkflowInstance*>
+listDefinitionNames() : vector<string>
+definitions() : const DefinitionRepository&
+setMigrationPolicy(policy : shared_ptr<IMigrationStrategy>)
+recovery() : RecoveryManager&
+instances() : InstanceManager&
WorkflowEvent
-type : EventType
-sourceId : string
-instanceId : string
-payload : WorkflowData
+WorkflowEvent(type : EventType, sourceId : const string&, instanceId : const string&, payload : const WorkflowData&)
+getType() : EventType
+getSourceId() : const string&
+getInstanceId() : const string&
+getPayload() : const WorkflowData&
<<enumeration>>
EventType
WORK_ITEM_STATE_CHANGED
DEADLINE_EXPIRED
THRESHOLD_BREACHED
EXTERNAL_EVENT
INTEGRATION_FAILED
ACCESS_DENIED
Condition
-key : string
-expectedValue : string
+evaluate(data : const WorkflowData&) : bool
+Condition(key : const string&, expectedValue : const string&)
VersionManager
-policy : MigrationStrategy*
-instances : InstanceManager&
-recovery : RecoveryManager&
+setPolicy(p : MigrationStrategy*)
+onNewVersion(def : const WorkflowDefinition&)
+VersionManager(instances : InstanceManager&, recovery : RecoveryManager&)
+getPolicy() : MigrationStrategy*
+resolve(repo : const DefinitionRepository&, name : const string&, version : int) : const WorkflowDefinition*
<<Interface>>
DefinitionRepository
+save(def : WorkflowDefinition*)
+find(name : string, version : int) : const WorkflowDefinition*
+latest(name : string) : const WorkflowDefinition*
+names() : vector<string>
+versionsOf(name : const string&) : vector<int>
ParticipantDirectory
-participants : map<string, Participant*>
-roles : map<string, Role*>
+add(p : Participant*)
+findByRole(r : const Role&) : vector<Participant*>
+find(id : string) : Participant*
+ParticipantDirectory()
+addRole(role : Role*)
owns
WorkflowInstance
-id : string
-outcome : WorkflowOutcome
-definition : const WorkflowDefinition*
-root : unique_ptr<WorkNode>
-data : WorkflowData
-history : ActionHistory
-started : bool
+getId() : string
+start()
+getRoot() : WorkNode&
+getData() : WorkflowData&
+getHistory() : ActionHistory&
+setOutcome(o : WorkflowOutcome)
+rebindTo(def : const WorkflowDefinition*)
+createSnapshot() : InstanceSnapshot*
+restore(s : const InstanceSnapshot&)
+WorkflowInstance(id : const string&, def : const WorkflowDefinition&, input : const WorkflowData&)
+getDefinition() : const WorkflowDefinition&
+getDefinitionName() : string
+getDefinitionVersion() : int
+isStarted() : bool
+getOutcome() : WorkflowOutcome
+isFinished() : bool
InstanceManager
-instances : map<string, WorkflowInstance*>
-order : vector<string>
-counter : long
+createInstance(def : const WorkflowDefinition&, input : WorkflowData) : WorkflowInstance*
+find(id : string) : WorkflowInstance*
+runningInstancesOf(def : const WorkflowDefinition&) : vector<WorkflowInstance*>
+get(id : string&) : WorkflowInstance&
+runningInstancesOf(name : const string&) : vector<WorkflowInstance*>
+all() : vector<WorkflowInstance*>
+count() : size_t
-nextId() : string
clones template of
created from (version)
InMemoryDefinitionRepository
-store : map<string, vector<WorkflowDefinition*>>
-attribute : map<string, VersionList>
+save(def : WorkflowDefinition*)
+find(name : string, version : int) : const WorkflowDefinition*
+latest(name : string) : const WorkflowDefinition*
+InMemoryDefinitionRepository()
+names() : vector<string>
+versionsOf(name : const string&) : vector<int>
versions
Participant
-id : string
-workload : int
-skills : vector<string>
-roles : vector<Role*>
+getId() : string
+hasRole(r : const Role&) : bool
+getWorkload() : int
+getSkills() : const vector<string>&
+Participant(id : const string&)
+hasSkill(skill : const string&) : bool
+addRole(role : Role*)
+addSkill(skill : const string&)
+adjustWorkload(delta : int)
holds
Role
-name : string
+Role(name : const string&)
+getName() : const string&
owns
WorkflowDefinition
-name : string
-version : int
-published : bool
-root : unique_ptr<WorkNode>
-escalation : shared_ptr<EscalationHandler>
-approval : shared_ptr<ApprovalHandler>
+getRoot() : const WorkNode&
+getVersion() : int
+getEscalationChain() : EscalationHandler*
+getApprovalChain() : ApprovalHandler*
+publish()
+WorkflowDefinition(name : const string&, root : unique_ptr<WorkNode>)
+getName() : const string&
+isPublished() : bool
+setEscalationChain(head : shared_ptr<EscalationHandler>)
+setApprovalChain(head : shared_ptr<ApprovalHandler>)
+setVersion(version : int)
+label() : string
-requireDraft(what : const char*)
cloned root
ActionHistory
-commands : vector<WorkflowCommand*>
-lines : vector<string>
+append(cmd : WorkflowCommand*)
+appendNote(text : string)
+entries() : const vector<WorkflowCommand*>&
+log() : const vector<string>&
+size() : size_t
+empty() : bool
WorkflowData
-values : map<string, string>
+get(key : string) : string
+set(key : string, value : string)
+has(key : string) : bool
+WorkflowData()
+getOr(key : const string&, fallback : const string&) : string
+remove(key : const string&) : bool
+keys() : vector<string>
+size() : size_t
+empty() : bool
<<enumeration>>
WorkflowOutcome
RUNNING
COMPLETED
REJECTED
CANCELLED
FAILED


Work Structure (Composite, Iterator, Visitor, Prototype, Builder)
WorkContainer
#children : vector<WorkNode*>
#routing : RoutingStrategy*
#routingKey : string
#nextToStart : size_t
+add(n : WorkNode*)
+remove(n : WorkNode*)
+getChildren() : const vector<WorkNode*>&
+onChildCompleted(child : WorkNode&)
+createLevelOrderIterator() : WorkNodeIterator*
+replaceChild(oldNode : WorkNode*, replacement : WorkNode*)
+isComplete() : bool
+setRouting(r : shared_ptr<RoutingStrategy>)
+getRouting() : shared_ptr<RoutingStrategy>
+setRoutingKey(key : const string&)
+getRoutingKey() : const string&
+findById(nodeId : const string&) : WorkNode*
+captureState(out : map<string, string>&)
+restoreState(in : const map<string, string>&)
children
template root
0..*
1
WorkNode
#id : string
#name : string
#parent : WorkContainer*
+start()
+cancel()
+isComplete() : bool
+clone() : WorkNode*
+accept(v : WorkNodeVisitor&)
+createIterator() : WorkNodeIterator*
+createReadyIterator() : WorkNodeIterator*
+getId() : string
+findById(nodeId : const string&) : WorkNode*
+captureState(out : map<string, string>&)
+restoreState(in : const map<string, string>&)
+getName() : const string&
+getParent() : WorkContainer*
Stage
+start()
+cancel()
+isComplete() : bool
+clone() : Stage*
+accept(v : WorkNodeVisitor&)
+createIterator() : WorkNodeIterator*
+Stage(id : const string&, name : const string&)
+createReadyIterator() : WorkNodeIterator*
SubProcess
-localData : WorkflowData
-outcome : WorkflowOutcome
+start()
+cancel()
+isComplete() : bool
+clone() : SubProcess*
+accept(v : WorkNodeVisitor&)
+createIterator() : WorkNodeIterator*
+SubProcess(id : const string&, name : const string&)
+onChildCompleted(child : WorkNode&)
+createReadyIterator() : WorkNodeIterator*
+getLocalData() : WorkflowData&
+getOutcome() : WorkflowOutcome
+setOutcome(o : WorkflowOutcome)
#routingData() : const WorkflowData&
creates
accept
WorkNodeIterator
+hasNext() : bool
+next() : WorkNode*
WorkNodeVisitor
+visit(s : Stage&)
+visit(p : SubProcess&)
+visit(w : WorkItem&)
ReadyItemIterator
-inner : DepthFirstIterator
-root : const WorkNode*
-buffered : WorkNode*
-readyPredicate : function<bool(const WorkItem&)>
+hasNext() : bool
+next() : WorkNode*
-fillBuffer()
+ReadyItemIterator(root : const WorkNode*)
+ReadyItemIterator(root : const WorkNode*, predicate : function<bool(const WorkItem&)>)
LevelOrderIterator
-pending : queue<WorkNode*>
+hasNext() : bool
+next() : WorkNode*
+LevelOrderIterator(root : WorkNode*)
DepthFirstIterator
-pending : stack<WorkNode*>
-root : const WorkNode*
+hasNext() : bool
+next() : WorkNode*
+DepthFirstIterator(root : const WorkNode*)
BottleneckVisitor
#durationSeconds : function<bool(const WorkItem&,double&)>
#observed : vector<pair<double, string>>
+visit(s : Stage&)
+visit(p : SubProcess&)
+visit(w : WorkItem&)
+slowestItems(n : int) : vector<string>
+BottleneckVisitor()
+BottleneckVisitor(source : function<bool(const WorkItem&,double&)>)
ProgressVisitor
-total : size_t
-completed : size_t
+visit(s : Stage&)
+visit(p : SubProcess&)
+visit(w : WorkItem&)
+percentComplete() : double
CycleTimeVisitor
-durationSeconds : function<bool(const WorkItem&,double&)>
-totalSeconds : double
-count : long
+visit(s : Stage&)
+visit(p : SubProcess&)
+visit(w : WorkItem&)
+averageSeconds() : double
+CycleTimeVisitor()
+CycleTimeVisitor(source : function<bool(const WorkItem&,double&)>)
DefinitionValidationVisitor
-problems : vector<string>
-seenIds : unordered_set<string>
+errors() : vector<string>
+visit(s : Stage&)
+visit(p : SubProcess&)
+visit(w : WorkItem&)
+DefinitionValidationVisitor()
-validateId(id : const string&)
WorkflowDirector
+construct(b : WorkflowDefinitionBuilder&)
directs
InteractiveDefinitionDirector
-steps : vector<WorkflowData>
+construct(b : WorkflowDefinitionBuilder&)
+InteractiveDefinitionDirector()
+addStep(step : const WorkflowData&)
+clear()
+stepCount() : size_t
DefinitionFileReader
-path : string
+construct(b : WorkflowDefinitionBuilder&)
+DefinitionFileReader(path : const string&)
-tokenize(line : const string&) : vector<string>
+dispatch(b : WorkflowDefinitionBuilder, tokens : const vector<string>&)
WorkflowDefinitionBuilder
+beginStage(name : string)
+beginSubProcess(name : string)
+endContainer()
+addHumanTask(id : string, role : string)
+addAutomatedTask(id : string, connector : string)
+dependsOn(id : string, prereqId : string)
+setRouting(key : string)
+setAssignment(id : string, key : string)
+setDeadline(id : string, seconds : int)
+decorateWith(id : string, key : string)
+addEscalationLevel(role : string, seconds : int)
+addApprovalLevel(role : string, limits : ApprovalLimits)
+build() : WorkflowDefinition*
+beginDefinition(name : const string&)
Implements every WorkflowDefinitionBuilder operation, build() validates with DefinitionValidationVisitor.
DefinitionBuilder
-open : stack<WorkContainer*>
-root : Stage
-definitionName : string
-escalationHead : shared_ptr<EscalationHandler>
-escalationTail : EscalationHandler*
-approvalHead : shared_ptr<ApprovalHandler>
-approvalTail : ApprovalHandler*
-nextAutoId : int
-usedIds : set<string>
-dependencies : vector<pair<string, string>>
-assignments : map<string, string>
-deadlines : map<string, int>
-decorations : map<string, vector<string>>
+build() : WorkflowDefinition*
+abort()
-requireNode(id : const string&) : WorkNode*
-requireProduct()
-checkIdFree(id : const string&)
-newAutoId(prefix : const char*) : string
-validate()
-reset()
validates with
creates
escalati
candidates from

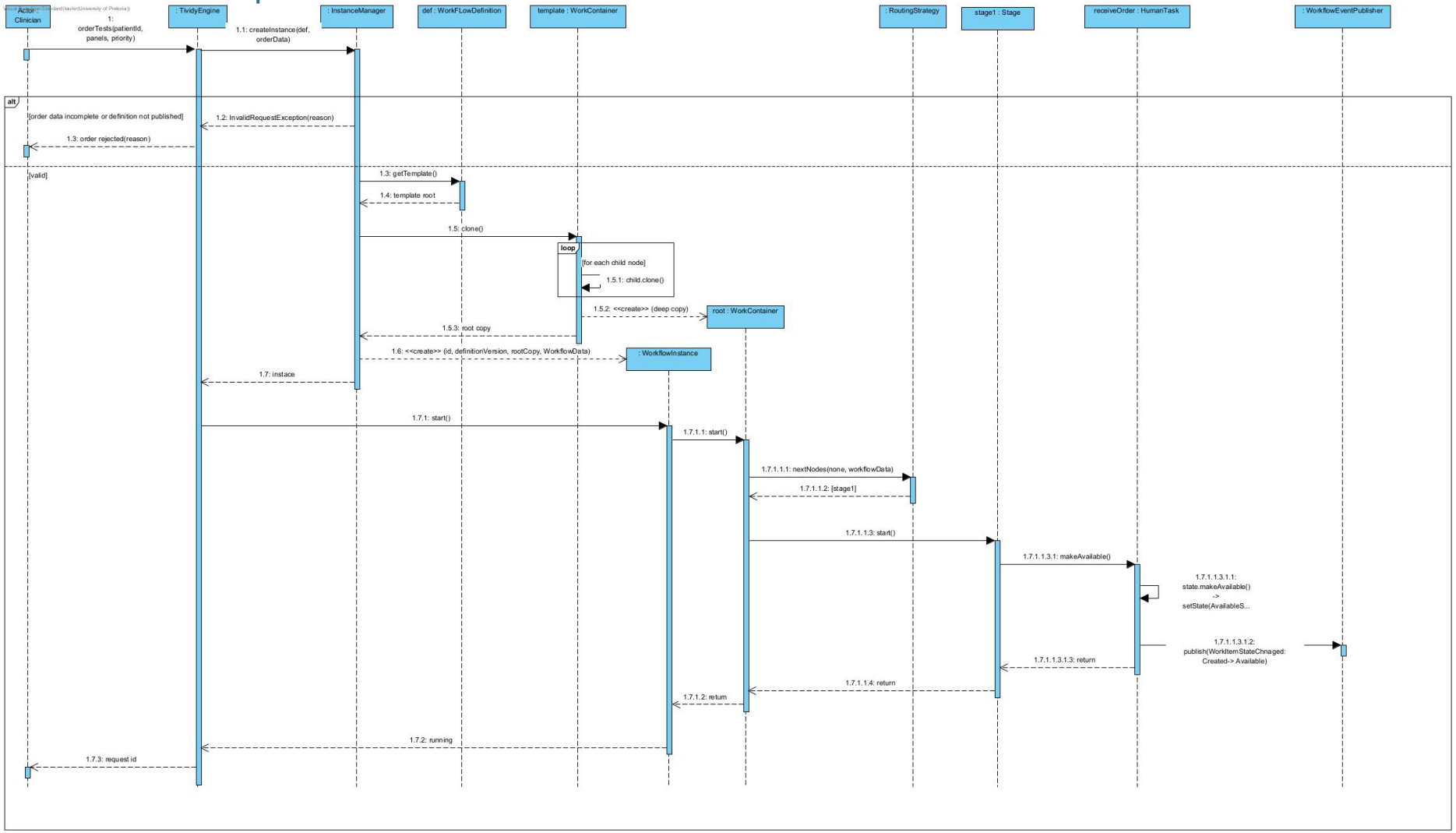

Events and Surroundings (Observer, Mediator, Adaptor)
<<Interface>>
ResourceMediator
+reserve(requester : Task&, type : string, priority : int) : Resource*
+release(r : Resource*)
ResourceCoordinator
-waiting : priority_queue<ResourceRequest>
-resources : vector<Resource*>
+onResourceFailed(r : Resource&)
+reserve(requester : Task&, type : string, priority : int) : Resource*
+release(r : Resource*)
+addResource(r : Resource*)
WorkflowEventPublisher
-listeners : map<EventType, vector<WorkflowEventListener*>>
+subscribe(t : EventType, l : WorkflowEventListener*)
+unsubscribe(t : EventType, l : WorkflowEventListener*)
+publish(e : const WorkflowEvent&)
informs
waiting
manages
Resource
-id : string
-type : string
-inUse : bool
+isAvailable() : bool
+reportFailure()
+Resource(id : const string&, type : const string&)
+getId() : const string&
+getType() : const string&
ResourceRequest
-requester : Task*
-type : string
-priority : int
+ResourceRequest(requester : Task*, type : const string&, priority : int)
+getRequester() : Task*
+getType() : const string&
+getPriority() : int
<<Interface>>
WorkflowEventListener
+onEvent(e : const WorkflowEvent&)
ConsumableResource
+ConsumableResource(id : const string&)
PersonnelResource
+PersonnelResource(id : const string&)
EquipmentResource
+EquipmentResource(id : const string&)
ConnectorRegistry
-connectors : map<string, ExternalConnector*>
+add(name : string, c : ExternalConnector*)
+find(name : string) : ExternalConnector*
EventTriggerListener
-triggers : map<string, string>
+onEvent(e : const WorkflowEvent&)
NotificationListener
-channel : string
+onEvent(e : const WorkflowEvent&)
+NotificationListener(channel : const string&)
MetricsCollector
+onEvent(e : const WorkflowEvent&)
AuditLogger
+onEvent(e : const WorkflowEvent&)
DependencyResolver
-instance : WorkflowInstance&
+onEvent(e : const WorkflowEvent&)
+DependencyResolver(instance : WorkflowInstance&)
+unlockReadyTasks()
<<Interface>>
ExternalConnector
+send(request : const WorkflowData&) : WorkflowData
+isAvailable() : bool
Integrations (Adapter: concrete adaptors for third-party interfaces)
EmailAdapter
-service : SmtpMailService*
+send(request : const WorkflowData&) : WorkflowData
+isAvailable() : bool
+EmailAdapter(service : SmtpMailService*)
SmsGatewayAdapter
-service : SmsGatewayService*
+send(request : const WorkflowData&) : WorkflowData
+isAvailable() : bool
+SmsGatewayAdapter(service : SmsGatewayService*)
RestApiAdapter
-client : ThirdPartyHttpClient*
+send(request : const WorkflowData&) : WorkflowData
+isAvailable() : bool
+RestApiAdapter(client : ThirdPartyHttpClient*)
DatabaseAdapter
-driver : LegacyDatabaseDriver*
+send(request : const WorkflowData&) : WorkflowData
+isAvailable() : bool
+DatabaseAdapter(driver : LegacyDatabaseDriver*)
FileExportAdapter
-service : FileStorageService*
+send(request : const WorkflowData&) : WorkflowData
+isAvailable() : bool
+FileExportAdapter(service : FileStorageService*)
adaptee
SmtpMailService
+sendMail(to : string, subject : string, html : string) : int
SmsGatewayService
+sendSms(number : string, text : string) : int
ThirdPartyHttpClient
+doRequest(method : string, url : string, body : string) : string
LegacyDatabaseDriver
+executeQuery(sql : string) : string
FileStorageService
+writeFile(path : string, bytes : string) : bool


Rules and Decisions (Strategy, Chain of Responsibility)
«interface»
RoutingStrategy
+matchNode(node : WorkNode&, c : WorkContext&, d : const WorkflowData&) : vector<WorkNode*>
+isComplete(c : const WorkContext&) : bool
ConditionalRouting
-branches : vector<pair<Condition, string>>
+matchNode(node : WorkNode&, c : WorkContext&, d : const WorkflowData&) : vector<WorkNode*>
+isComplete(c : const WorkContext&) : bool
parallelRouting
+matchNode(node : WorkNode&, c : WorkContext&, d : const WorkflowData&) : vector<WorkNode*>
+isComplete(c : const WorkContext&) : bool
SequentialRouting
+matchNode(node : WorkNode&, c : WorkContext&, d : const WorkflowData&) : vector<WorkNode*>
+isComplete(c : const WorkContext&) : bool
LoopRouting
-repeatWhile : Condition
-maxIterations : int
+matchNode(node : WorkNode&, c : WorkContext&, d : const WorkflowData&) : vector<WorkNode*>
+isComplete(c : const WorkContext&) : bool
EscalationHandler
-next : EscalationHandler*
+setNext(h : EscalationHandler*)
+handle(r : EscalationRequest&)
next
EscalationRequest
-level : string
-deadlineSeconds : int
-escalatedAgain : bool
-handler : EscalationHandler&
TimeoutEscalationLevel
-handler : EscalationHandler&
ApprovalHandler
-next : ApprovalHandler*
+setNext(h : ApprovalHandler*)
+handle(r : ApprovalRequest&)
next
ApprovalRequest
-stepId : string
-result : WorkflowData
-decision : string
-comment : string
+getStepId() : const string&
+getDecision() : const string&
+getResult() : const WorkflowData&
+setResult(decision : const string&, reason : const string&)
ApprovalLevel
-role : string
-limit : ApprovalLimit
+handle(r : ApprovalRequest&) : bool
ApprovalLimits
-maxPriority : int
-allowedCategories : set<string>
+covers(r : const ApprovalRequest&) : bool
AssignmentStrategy
«interface»
AssignmentStrategy
+selectParticipant(item : const Task&, c : const vector<Participant*>&) : Participant*
RoundRobinAssignment
-lastIndex : int
+selectParticipant(item : const Task&, c : const vector<Participant*>&) : Participant*
SkillAssignment
+selectParticipant(item : const Task&, c : const vector<Participant*>&) : Participant*
CapacityAssignment
+selectParticipant(item : const Task&, c : const vector<Participant*>&) : Participant*
RuleAssignment
+selectParticipant(item : const Task&, c : const vector<Participant*>&) : Participant*
DirectAssignment
+selectParticipant(item : const Task&, c : const vector<Participant*>&) : Participant*
Actions and Recovery (Command, Memento)
CommandProcessor
+execute(cmd : WorkflowCommand*, ctx : WorkflowInstance&)
+undo() noexcept : WorkflowInstance&
+canUndo() const : bool
+redo() : WorkflowInstance&
executes
WorkflowCommand
#time : Participant*
#timestamp : time_t
-undone : bool
+execute()
+undo()
+canUndo() : bool
+describe() : string
+getId() const : string
+getActor() : Participant*
+getTimestamp() : time_t
+isUndone() : bool
#backup(ctx) : string
RejectCommand
-reason : string
+execute()
+describe() : string
AssignCommand
-assignee : Participant*
-previous : Participant*
+execute()
+undo()
+canUndo() : bool
+describe() : string
CompleteCommand
-result : WorkflowData
+execute()
+describe() : string
EscalateCommand
-reason : string
+execute()
+describe() : string
ApproveCommand
-details : WorkflowData
+execute()
+describe() : string
StartCommand
+execute()
+describe() : string
CancelRequestCommand
-steps : vector<WorkflowCommand*>
+addStep(step : WorkflowCommand*)
+execute()
+describe() : string
steps
RecoveryManager
-checkpoints : map<string, string>
+createSnapshot(ctx : WorkflowInstance&, label : string)
+rollback(id : WorkflowInstance&, label : string)
+discard(ctx : const WorkflowInstance&, label : string)
+hasCheckpoint(ctx : const WorkflowInstance&, label : string) : bool
+listCheckpoints(ctx : const WorkflowInstance&) : vector<string>
+size() : size_t
-key(ctx : const WorkflowInstance&, label : string) : string
checkpoints
InstanceSnapshot
-itemStates : map<string, string>
-assignees : map<string, string>
-data : WorkflowData
-times : vector<TaskTime>
-createdAt : time_t
+capture(ctx)
+restore(ctx)
+byteSize() : size_t
Private constructor and state: WorkflowInstance is a C++ friend
so only the originator can read it

«interface»
ResiliencePolicy
+isFailure(t : IntegrationTask&, a : Attempt) : bool
«interface»
MigrationStrategy
+migrate(inst : WorkflowInstance&, def : const WorkflowDefinition&, rm : RecoveryManager&)
FallbackPolicy
-fallbackConnector : string
+isFailure(t : IntegrationTask&, a : Attempt) : bool
RetryPolicy
-maxAttempts : int
+isFailure(t : IntegrationTask&, a : Attempt) : bool
CompleteThenUpdate
+migrate(inst : WorkflowInstance&, def : const WorkflowDefinition&, rm : RecoveryManager&)
ForwardRecovery
+migrate(inst : WorkflowInstance&, def : const WorkflowDefinition&, rm : RecoveryManager&)
DynamicMigration
+migrate(inst : WorkflowInstance&, def : const WorkflowDefinition&, rm : RecoveryManager&)
BackwardRecovery
+migrate(inst : WorkflowInstance&, def : const WorkflowDefinition&, rm : RecoveryManager&)

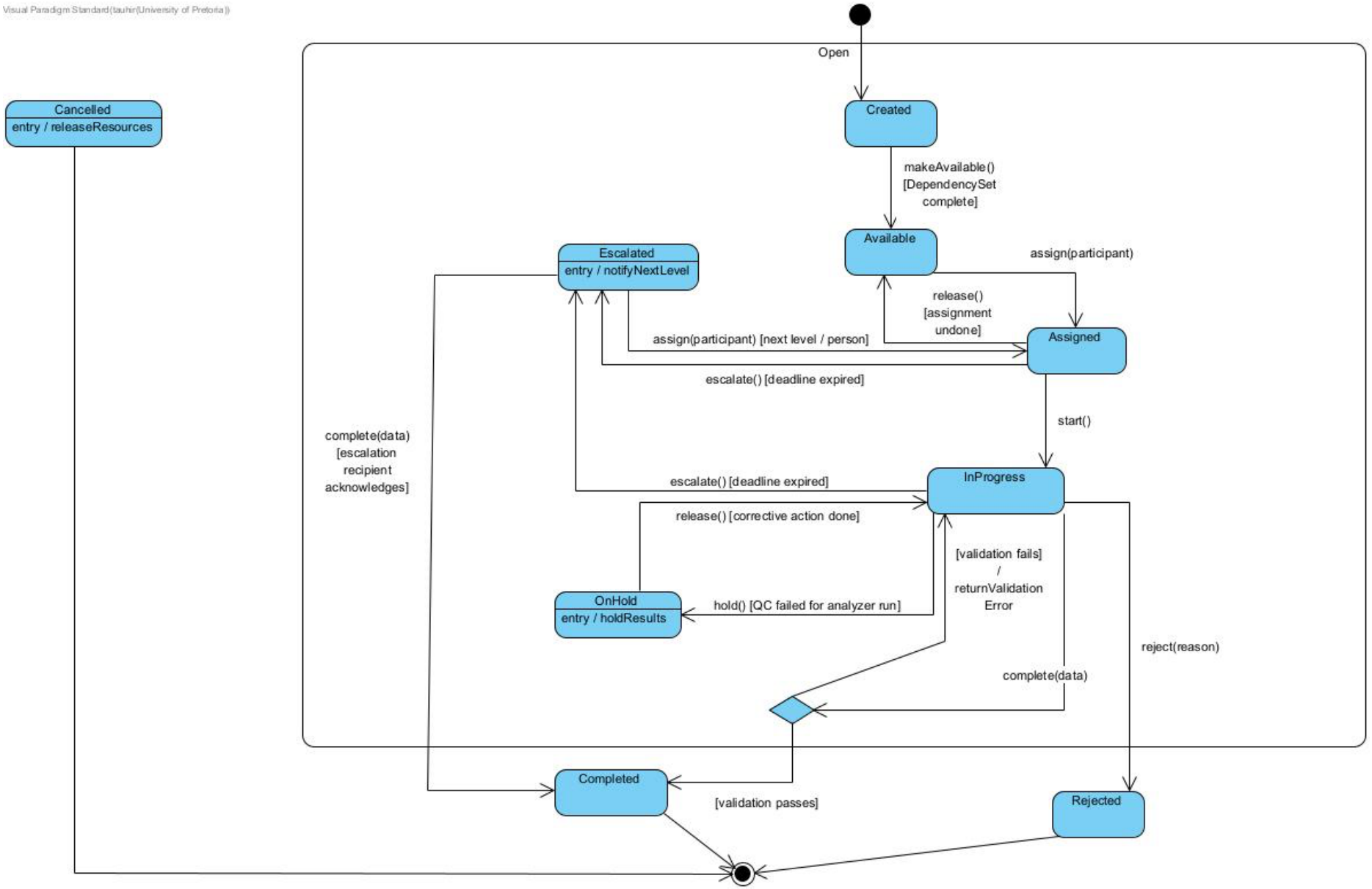
Task 6: Sequence & State diagrams

Create a start request

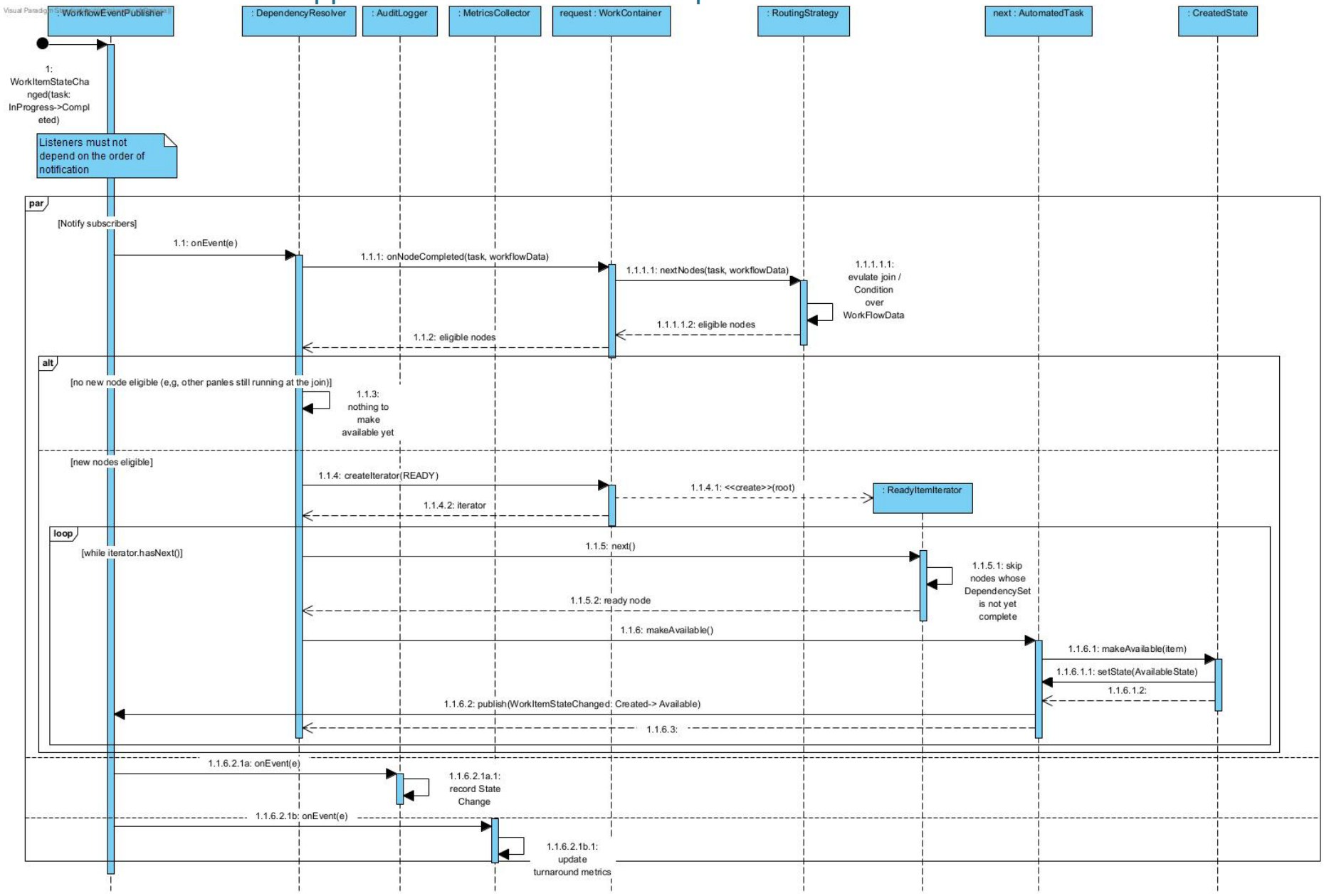


Work item lifecycle

Visual Paradigm Standard(tauhir(University of Pretoria))



Determine what happens after a work item completes



Build Errors, Causes and Fixes

Spelling error in WorkflowCommand.h

```
(base) hannahdiedrick@Hannahs-MacBook-Air-2 Prac6_FInalProject % make -f Makefile
g++ -std=c++17 -Wall -Wextra -pedantic -g -c ActionHistory.cpp -o build/ActionHistory.o
In file included from ActionHistory.cpp:13:
./WorkflowCommand.h:39:41: error: unknown type name 'Participant'; did you mean 'Particpant'?
   39 |     WorkflowCommand(WorkItem* receiver, Participant* actor);
      |                                         ^~~~~~~~~~~
      |                                         Particpant
./WorkflowCommand.h:15:7: note: 'Particpant' declared here
   15 | class Particpant;
      |       ^
./WorkflowCommand.h:62:5: error: unknown type name 'Participant'; did you mean 'Particpant'?
   62 |     Participant* getActor() const;
      |     ^~~~~~~~~~~
      |     Particpant
./WorkflowCommand.h:15:7: note: 'Particpant' declared here
   15 | class Particpant;
      |       ^
./WorkflowCommand.h:74:5: error: unknown type name 'Participant'; did you mean 'Particpant'?
   74 |     Participant* actor;      // Not owned. nullptr when the system acts.
      |     ^~~~~~~~~~~
      |     Particpant
./WorkflowCommand.h:15:7: note: 'Particpant' declared here
   15 | class Particpant;
      |       ^
3 errors generated.
```

Cause: The forward declaration at the top of the file was spelt “class Particpant”. The rest of the file used the correct spelling, so the compiler had never heard of Participant.

Fix: Corrected the forward declaration to “class Participant”. All three errors disappeared because they had one cause.

Include path, filename case and missing header

```
In file included from ApprovalLimits.cpp:1:
./ApprovalLimits.h:2:10: warning: non-portable path to file '"person5Forward.h"'; specified path differs in case from file name on disk [-Wnonportable-include-path]
    2 | #include "Person5Forward.h"
      |          ^~~~~~~~~~~~~~~~~
      |          "person5Forward.h"
In file included from ApprovalLimits.cpp:2:
./ApprovalRequest.h:2:10: warning: non-portable path to file '"person5Forward.h"'; specified path differs in case from file name on disk [-Wnonportable-include-path]
    2 | #include "Person5Forward.h"
      |          ^~~~~~~~~~~~~~~~~
      |          "person5Forward.h"
In file included from ApprovalLimits.cpp:1:
./ApprovalLimits.h:6:9: warning: private field 'maxPriority_' is not used [-Wunused-private-field]
    6 |     int maxPriority_ = 0;
      |         ^
3 warnings generated.
g++ -std=c++17 -Wall -Wextra -pedantic -g -c ApprovalRequest.cpp -o build/ApprovalRequest.o
In file included from ApprovalRequest.cpp:1:
./ApprovalRequest.h:2:10: warning: non-portable path to file '"person5Forward.h"'; specified path differs in case from file name on disk [-Wnonportable-include-path]
    2 | #include "Person5Forward.h"
      |          ^~~~~~~~~~~~~~~~~
      |          "person5Forward.h"
./ApprovalRequest.h:7:19: warning: private field 'result_' is not used [-Wunused-private-field]
    7 |     WorkflowData* result_ = nullptr; // non-owning workflow data
      |                   ^
2 warnings generated.
g++ -std=c++17 -Wall -Wextra -pedantic -g -c ApproveCommand.cpp -o build/ApproveCommand.o
In file included from ApproveCommand.cpp:6:
./ApproveCommand.h:13:10: fatal error: 'WorkflowData.h' file not found
   13 | #include "WorkflowData.h"
      |          ^~~~~~~~~~~~~~~~
1 error generated.
make: *** [build/ApproveCommand.o] Error 1
```

Cause:

- The code said #include "Person5Forward.h" but the file on disk is person5Forward.h. macOS ignores the difference in capital letters, so it compiles there, but it fails on Linux and on most build servers.
- The unused fields belong to approval logic that was not yet finished, so nothing read them.
- WorkflowData.h belongs to the core engine module, which had not yet been merged into the shared branch, so no such file existed to include.

Fix:

- Include names now match filenames exactly, including capital letters. The temporary forward-declaration files were later deleted once the real headers existed.
- The unused-field warnings were left for the moment, because they do not affect behaviour. The fields are used once the approval logic is completed.
- The core engine files were merged into the shared branch, which supplied WorkflowData.h.

Class defined twice (WorkflowData and Participant)

```
In file included from ./WorkItem.h:5:
./Forward.h:18:7: error: redefinition of 'WorkflowData'
   18 | class WorkflowData {
      |       ^
./WorkflowData.h:12:7: note: previous definition is here
   12 | class WorkflowData {
      |       ^
In file included from ApproveCommand.cpp:11:
In file included from ./WorkItem.h:5:
./Forward.h:35:7: error: redefinition of 'Participant'
   35 | class Participant {
      |       ^
./Participant.h:4:7: note: previous definition is here
    4 | class Participant {
      |       ^
2 errors generated.
```

Cause: Forward.h was a shared header meant to hold only forward declarations. It had been given complete class bodies as stand-ins for classes that had not yet been delivered. When the real WorkflowData.h and Participant.h arrived, both definitions were visible at once and C++ allows only one.

Fix: Removed the stand-in class bodies from Forward.h so it contains only forward declarations such as class WorkflowData. Files that need the full class now include the real header.

Wrong arguments in BackwardRecovery.cpp

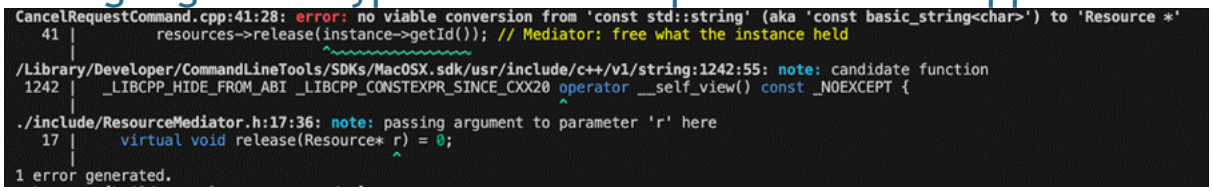
```
      |           "person5Forward.h"
BackwardRecovery.cpp:16:27: error: reference to type 'const WorkflowDefinition' could not bind to an rvalue of type 'const WorkflowDefinition *'
   16 |         instance.rebindTo(&definition);
      |                           ^~~~~~~~~~~
./WorkflowInstance.h:54:45: note: passing argument to parameter 'def' here
   54 |     void rebindTo(const WorkflowDefinition& def);
      |                                             ^
BackwardRecovery.cpp:19:41: error: too few arguments to function call, expected 2, have 1
   19 |         recovery.discard(checkpointLabel);
      |         ~~~~~~~~~~~~~~~~                ^
./RecoveryManager.h:55:10: note: 'discard' declared here
   55 |     void discard(const WorkflowInstance& inst, const std::string& label);
      |          ^       ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
BackwardRecovery.cpp:22:37: error: too few arguments to function call, expected 2, have 1
   22 |     recovery.discard(checkpointLabel);
      |     ~~~~~~~~~~~~~~~~                ^
./RecoveryManager.h:55:10: note: 'discard' declared here
   55 |     void discard(const WorkflowInstance& inst, const std::string& label);
      |          ^       ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
2 warnings and 3 errors generated.
```

Cause: Two interface mismatches.

- rebindTo takes a reference to a definition, but the code passed &definition, a pointer. The variable was already a reference, so the address-of operator was wrong.
- RecoveryManager::discard takes the instance and the checkpoint label, because two instances may use the same label. The code was written against the shorter signature that had only the label.

Fix: Changed the call to instance.rebindTo(definition) and the two discard calls to recovery.discard(instance, checkpointLabel).

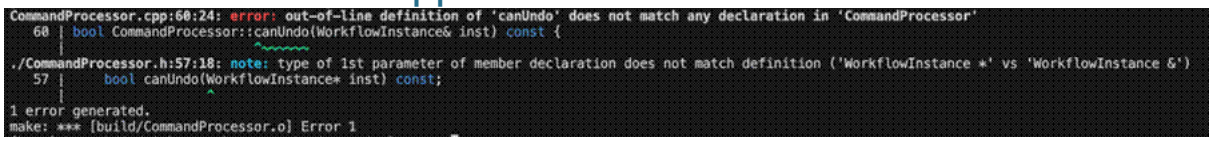
Wrong argument type in CancelRequestCommand.cpp



Cause: ResourceMediator::release accepts a pointer to the resource that was reserved. The code passed the instance id, a string, as if the mediator could find the resource by id.

Fix: The command now releases the Resource* that was returned when the resource was reserved for the instance, instead of the instance id.

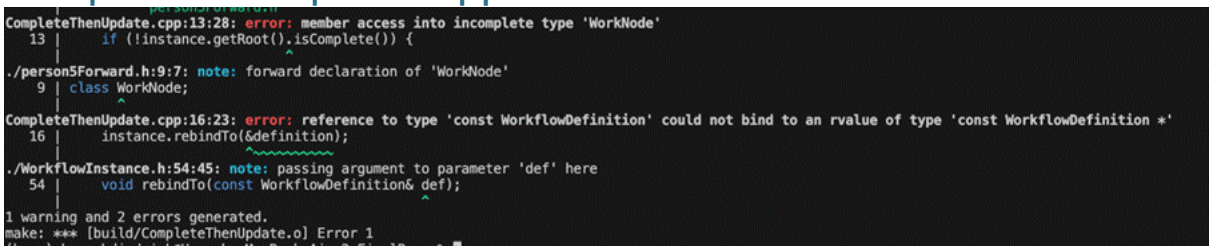
Declaration and definition do not match in CommandProcessor.cpp



Cause: The function was declared with a pointer parameter in the header and defined with a reference parameter in the source file. C++ treats these as two different functions.

Fix: Both now use a reference, which matches the rest of the CommandProcessor interface.

Incomplete type and wrong argument in CompleteThenUpdate.cpp

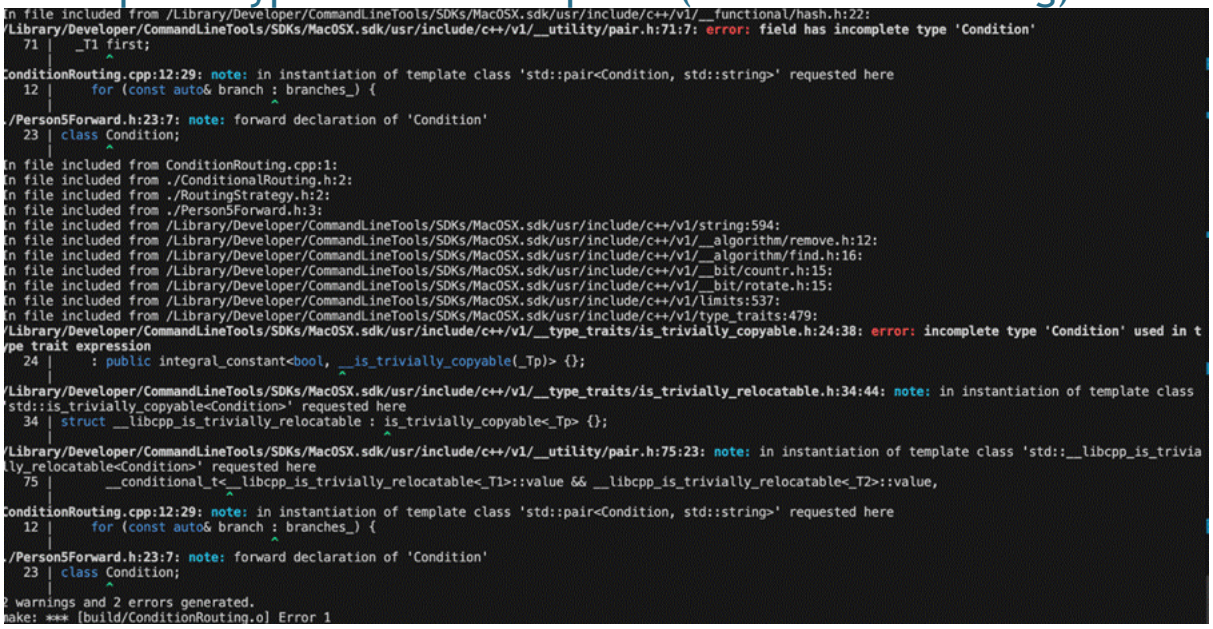


Cause:

- WorkNode was only forward-declared (in person5Forward.h). A forward declaration tells the compiler that a class exists but not what is in it, so calling isComplete() on it is impossible.
- The pointer was again passed where a reference is expected.

Fix: Added #include "WorkNode.h" to the .cpp file, where the member functions are called, and changed the call to instance.rebindTo(definition).

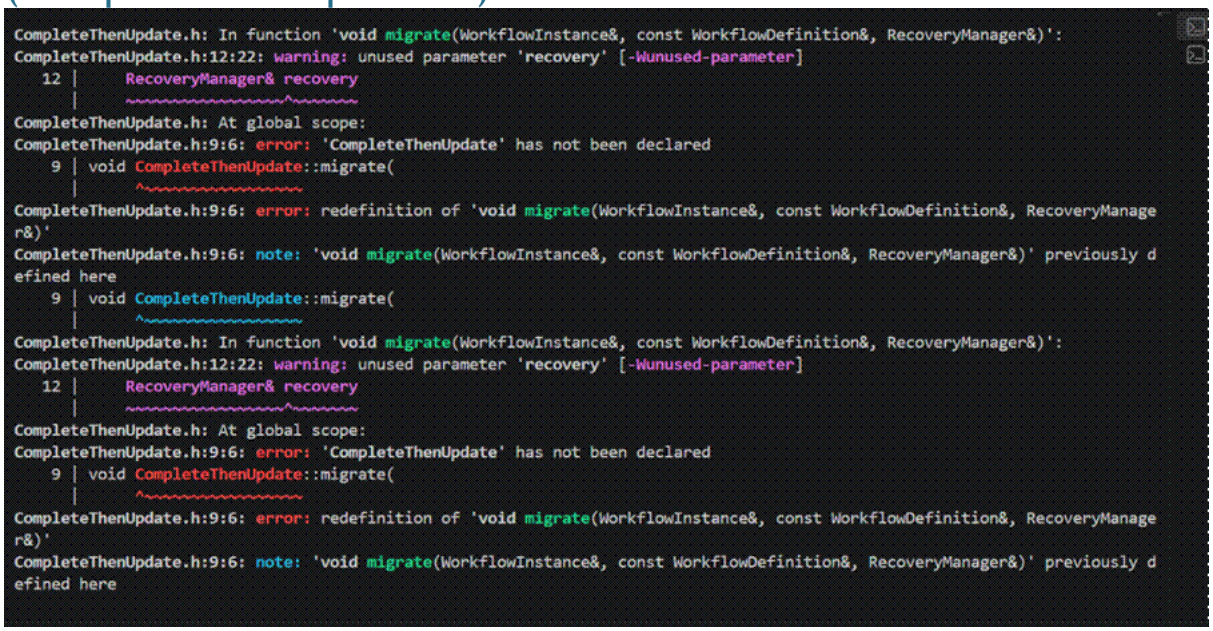
Incomplete type inside a template (ConditionalRouting)



Cause: ConditionalRouting keeps its branches in a collection of std::pair<Condition, std::string>. A pair stores its first member by value, so the compiler must know the full size and layout of Condition. The header only had a forward declaration of it.

Fix: Included the real Condition.h in ConditionalRouting.h in place of the forward declaration.

Function defined in the header without a class (CompleteThenUpdate.h)



Cause: The header contained the body of CompleteThenUpdate::migrate but not the class declaration that CompleteThenUpdate:: refers to. A function body in a header is also compiled into every file that includes it, which led to the redefinition error. The warning appeared because the body never used its recovery argument.

Fix: Restructured the file in the standard way. The header declares the class and the function (void migrate(...) override;), and the body lives in CompleteThenUpdate.cpp. The unused parameter is marked as intentionally unused.

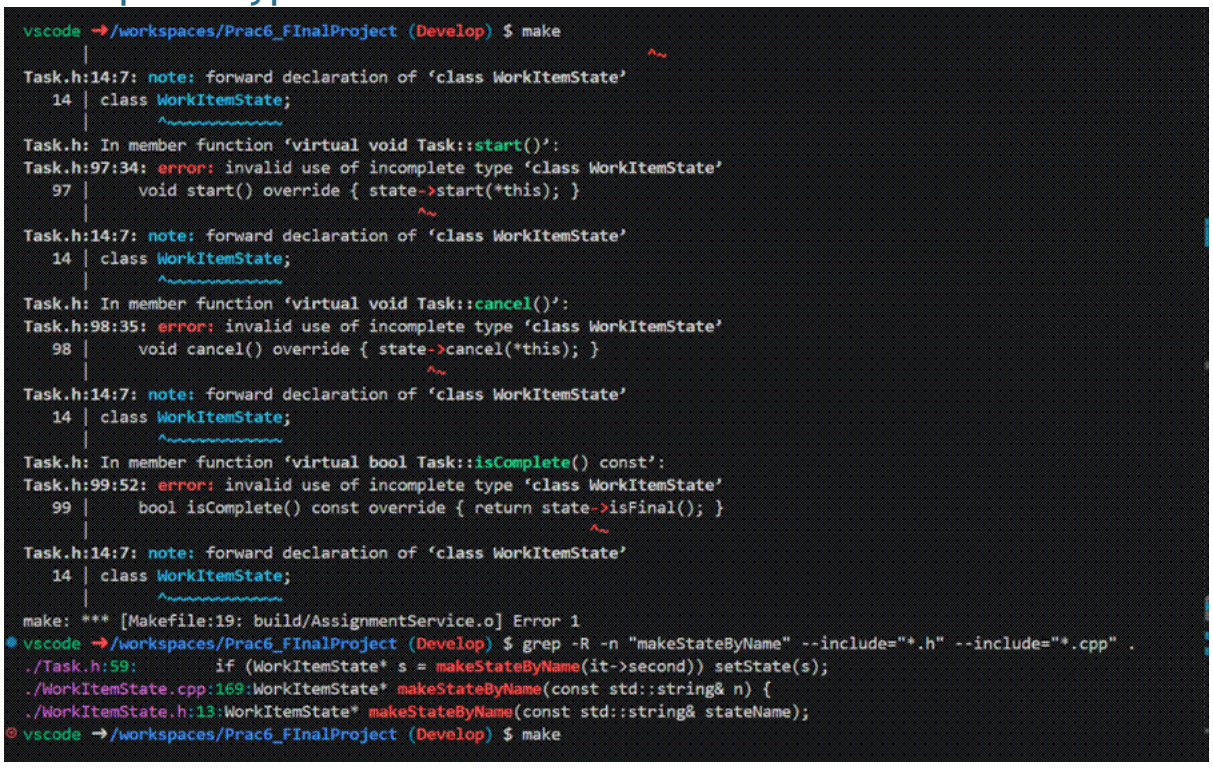
Linker error: undefined WorkflowInstance functions



Cause: A linker error is different from a compiler error. Every file compiled, but the linker could not find a body for the functions declared in WorkflowInstance.h. The file named WorkflowInstance.cpp had been overwritten with the code of InstanceSnapshot.cpp, so no WorkflowInstance:: function was defined anywhere.

Fix: Restored the correct WorkflowInstance.cpp. The two Memento functions, createSnapshot() and restore(), were rewritten to use the real fields of InstanceSnapshot. The project was rebuilt and the core engine tests were run to confirm that every symbol resolves and behaviour is unchanged. InstanceSnapshot.cpp was confirmed to exist in its own file.

Incomplete type WorkItemState in Task.h



Cause: Task.h forward-declares WorkItemState because the two classes depend on each other. The State pattern needs this: a Task holds a state and every state operation takes a Task. The short member functions were written inside the header, where the compiler can see only the forward declaration, so calling methods on the state failed. The team also confirmed that makeStateByName exists in WorkItemState.cpp, so the problem was visibility and not a missing function.

Fix: Moved the function bodies from Task.h into Task.cpp, which includes WorkItemState.h. The header keeps only the declarations and the forward declaration, and the circular include is avoided.

Incomplete type WorkItemState in AutomatedTask and IntegrationTask

```
Task.h:14:7: note: forward declaration of ‘class WorkItemState’
   14 | class WorkItemState;
      |       ^~~~~~~~~~~~~
Task.h: In member function ‘virtual void Task::cancel()’:
Task.h:98:35: error: invalid use of incomplete type ‘class WorkItemState’
   98 |     void cancel() override { state->cancel(*this); }
      |                                   ^~
Task.h:14:7: note: forward declaration of ‘class WorkItemState’
   14 | class WorkItemState;
      |       ^~~~~~~~~~~~~
Task.h: In member function ‘virtual bool Task::isComplete() const’:
Task.h:99:52: error: invalid use of incomplete type ‘class WorkItemState’
   99 |     bool isComplete() const override { return state->isFinal(); }
      |                                                    ^~
Task.h:14:7: note: forward declaration of ‘class WorkItemState’
   14 | class WorkItemState;
      |       ^~~~~~~~~~~~~
AutomatedTask.h: In member function ‘virtual void AutomatedTask::execute()’:
AutomatedTask.h:16:14: error: invalid use of incomplete type ‘class WorkItemState’
   16 |         state->complete(*this, WorkflowData());
      |              ^~
Task.h:14:7: note: forward declaration of ‘class WorkItemState’
   14 | class WorkItemState;
      |       ^~~~~~~~~~~~~
IntegrationTask.cpp: In member function ‘virtual void IntegrationTask::execute()’:
IntegrationTask.cpp:20:18: error: invalid use of incomplete type ‘class WorkItemState’
   20 |             state->complete(*this, response); // InProgress -> Completed
      |                  ^~
Task.h:14:7: note: forward declaration of ‘class WorkItemState’
   14 | class WorkItemState;
      |       ^~~~~~~~~~~~~
```

Cause: The same circular-dependency problem as error 11. AutomatedTask.h called a method on the state inside the header, and IntegrationTask.cpp did not include WorkItemState.h.

Fix: Moved the body of AutomatedTask::execute() into its .cpp file and added #include "WorkItemState.h" to both source files. Calls on the state now happen only where the full class is visible.

What this taught us:

- **Forward declarations are for pointers and references only.** If the code calls a method on a type, holds it by value, or uses it inside a template such as std::pair or std::vector, the full header must be included.

- **Put function bodies in .cpp files.** This avoids redefinition errors and lets two classes depend on each other without a circular include.
- **One definition per class.** Shared forward headers must contain only declarations. Placeholder classes must be deleted when the real class is merged.
- **Agree on function signatures once.** Pointer versus reference and the exact parameters. Where the code departs from the UML, both the UML and the headers are updated.
- **Declaration and definition must match exactly,** including const and the parameter types.
- **Filenames and include names must match exactly, including capitals,** so the code builds on macOS, Linux and any build server.
- **Build before every push, and read errors from the top.** Later messages are often side effects of the first. A linker error means a function body is missing from the build, not that the logic is wrong.

Assumptions Made For TIVIDY

1. Definitions and versions

A process must be published before it can run, and publishing locks it. The program rejects anything else with an error.

What the program assumes	What the user must do	What happens otherwise
Every process has a name and a root of work	Give each definition a non-empty name and a work tree	Creating it fails with an InvalidOperationException
A process is published before use	Publish the definition through the engine, which saves it and gives it a version	Starting an unpublished definition fails with an InvalidOperationException
A published definition never changes	To change a process, publish a new version under the same name	Editing a published definition (version, chains, publishing again) is refused
Names identify processes and are case-sensitive	Type the process name exactly as it was published	“lab” and “Lab” are different processes. An unknown name raises a NotFoundException
Approval and escalation chains belong to the definition	Attach them before publishing	After publishing, changing a chain is refused

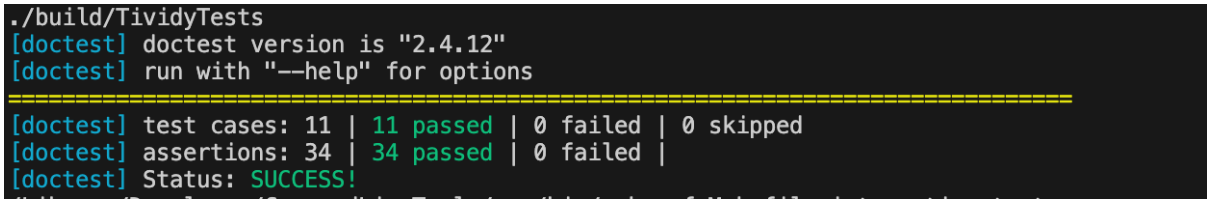
2. Starting and running work

Every run is a separate instance that the program names, and a finished instance is closed for good.

What the program assumes	What the user must do	What happens otherwise

Each start creates a new, independent instance	Start a workflow by its name, with its input data	Work cannot be added to an existing instance by starting again
The program names instances (WF-0001, WF-0002, ...)	Refer to an instance by its exact id	An unknown id raises a NotFoundException when a command is submitted. A plain lookup returns nothing
An instance starts once	Start it a single time	A second start fails with an InvalidOperationException
A finished instance is closed	Do not send commands to an instance that has ended	The command is refused with an InvalidOperationException
A final outcome cannot be replaced	Choose the right ending the first time	Changing a completed instance to another outcome is refused. Setting the same outcome again is ignored
Running instances stay on the version they started with	Set a migration policy if running work should move to a new version	Without one, old instances finish on their old version
A command must exist and belongs to the engine once submitted	Submit a real command and do not reuse or delete it afterwards	A null command is refused
A checkpoint belongs to one instance	Restore a checkpoint only into the instance that made it	Restoring it into another instance is refused

Testing: DocTest and Integration Testing:



```
/Library/Developer/CommandLineTools/usr/bin/make -f Makefile.integration test
./whole_project_demo --test

========== TEST: Healthcare - patient admission ==========
[PASS] DFS visits 1 root + 2 subprocesses + 4 tasks
[PASS] BFS traverses all nodes starting at root
[PASS] DFS and BFS give different valid traversal orders
[PASS] Exhausted iterator throws
[PASS] Definition validation rejects duplicate IDs
[PASS] Visitor validates unique, non-empty node IDs
[PASS] Initial progress is 0%
[PASS] No tasks ready before availability
[PASS] Ready filter returns all 4 available tasks
[PASS] Task transitions Available -> Assigned -> InProgress -> Completed
[PASS] Visitor reports 25% completion
[PASS] Ready filter excludes completed task
[PASS] Completed task rejects illegal hold
[PASS] Prototype makes independent nodes
[PASS] Prototype resets task execution state

=== Healthcare - patient admission : REPORTS ===
Progress (completed tasks): 25.0%
Average SAMPLE task duration: 675.0 seconds
Longest SAMPLE durations: H-TREAT H-TRIAGE
(Note: durations are scenario test fixtures, not measured runtime.)
Definition validation: PASS
Instance history entries: 1
  06:21:43  Instance H-RUN-01 created from Healthcare - patient admission v1
```

```
========== TEST: Logistics - order fulfilment ==========
[PASS] DFS visits 1 root + 2 subprocesses + 4 tasks
[PASS] BFS traverses all nodes starting at root
[PASS] DFS and BFS give different valid traversal orders
[PASS] Exhausted iterator throws
[PASS] Definition validation rejects duplicate IDs
[PASS] Visitor validates unique, non-empty node IDs
[PASS] Initial progress is 0%
[PASS] No tasks ready before availability
[PASS] Ready filter returns all 4 available tasks
[PASS] Task transitions Available -> Assigned -> InProgress -> Completed
[PASS] Visitor reports 25% completion
[PASS] Ready filter excludes completed task
[PASS] Completed task rejects illegal hold
[PASS] Prototype makes independent nodes
[PASS] Prototype resets task execution state

=== Logistics - order fulfilment : REPORTS ===
Progress (completed tasks): 25.0%
Average SAMPLE task duration: 270.0 seconds
Longest SAMPLE durations: L-PICK L-DISPATCH
(Note: durations are scenario test fixtures, not measured runtime.)
Definition validation: PASS
Instance history entries: 1
  06:21:43  Instance L-RUN-01 created from Logistics - order fulfilment v1
```

```
========== INTEGRATION: Healthcare - patient admission ==========
[PASS] Concrete task found through composite tree
[PASS] Strategy: role-based assignment chooses qualified participant
[PASS] Strategy: skills-based assignment chooses qualified participant
[PASS] Memento: saved checkpoint
[PASS] Command: assign changes state
[PASS] History: command appended
[PASS] Recovery: rollback restores Available
[PASS] History: rollback recorded
[PASS] Recovery: discarded checkpoint
[PASS] Command: start changes state
[PASS] Command: completion reaches terminal state
[PASS] Command history retains successful actions
[PASS] Illegal transition rejected through command processor
[PASS] History: failure recorded
[PASS] Visitor: progress reflects command-driven completion

========== INTEGRATION: Logistics - order fulfilment ==========
[PASS] Concrete task found through composite tree
[PASS] Strategy: role-based assignment chooses qualified participant
[PASS] Strategy: skills-based assignment chooses qualified participant
[PASS] Memento: saved checkpoint
[PASS] Command: assign changes state
[PASS] History: command appended
[PASS] Recovery: rollback restores Available
[PASS] History: rollback recorded
[PASS] Recovery: discarded checkpoint
[PASS] Command: start changes state
[PASS] Command: completion reaches terminal state
[PASS] Command history retains successful actions
[PASS] Illegal transition rejected through command processor
[PASS] History: failure recorded
[PASS] Visitor: progress reflects command-driven completion

TOTAL: 60 passed, 0 failed
```

Repository Link:

https://github.com/u25044215-Hannah/Prac6_FInalProject

Bibliography

Atlassian. (2024). Project dependencies [types & strategies] .

Finio, M. (2025). What is workflow orchestration?

FlowWright. (2026, March 2). 3 Simple workflow task routing concepts.

GeeksforGeeks. (2024). Error handling in EventDriven architecture.

IBM. (2025). IBM OpenPages.

SAP. (2026). What is workflow management: A beginner's guide.

Science Direct. (n.d.). Workflow management system - an overview.

Sedha, A. (2025, 10 28). Understanding workflow architecture in java: A complete guide.

Splashtop. (2024). Workflow management: Definitions, elements, benefits & more.

team, G. C. (2024). Best practices to optimize workitem assignment.

Tmssoftware.com. (2025). Basic concepts | TMS workflow studio documentation.

Unstructured. (2026). Managing dependencies in data workflows: A practical guide.